UC Berkeley CS170《Efficient Algorithms and Intractable Problems》学习笔记

> **课程**：CS 170 — Efficient Algorithms and Intractable Problems，UC Berkeley EECS **学期**：Fall 2026（授课教师 Nika Haghtalab、John Wright；来源：课程主页与 Lecture 1 讲义封面） **主页**：https://cs170.org/ **日程**：https://cs170.org/calendar/ **资源**：https://cs170.org/resources/ **答疑/讨论**：https://cs170.org/discussions/ **教材**：S. Dasgupta, C. H. Papadimitriou, U. V. Vazirani, *Algorithms*（简称 **DPV**），各讲义的 Readings 列均指向 DPV §0.2–§9 **数据来源**：本笔记全部技术内容来自 2026 年秋季学期公开的 18 份讲义 PDF（`/assets/lec/*.pdf`）、课程 Policies 页、Resources 页与 DPV 公开章节 PDF，抓取日期 2026-11-01。凡属我个人的推断、补充或"课程未明说"的内容，均显式标注"**推断**"。 **版权声明**：课程讲义、作业、讨论课材料的版权归 UC Berkeley 及各位作者所有（页面版权行为 "Copyright ©2026, Regents of the University of California and respective authors."）。本笔记为个人学习整理，仅摘录概念、定理与算法思想并用自己的语言重写证明与伪代码，不复制讲义原文、不转载题目原文；引用处均给出 URL。请勿将本笔记用于任何商业用途或再分发课程材料。 **阅读前提**：同目录 `UCB_CS70_离散数学与概率论_学习笔记.md` 已覆盖离散数学与概率论先修内容（归纳法、图论基础、条件概率、期望、方差、集中不等式）。本笔记**不重复**这些基础，只在用到时以「CS70 §…」的方式指回，把篇幅留给算法设计与分析本身。

1. 课程概览
   1.1 这门课到底在教什么
   1.2 先修与前置假设
   1.3 评分构成（Fall 2026，来自 Policies 页）
   1.4 八章讲义地图（Lecture 1–18 → 知识块）
   1.5 与 CS70 / MIT 6.006 的关系
2. 第 1 章：渐近分析与算法分析的写法
   2.1 问题动机
   2.2 定义
   2.3 证明模板（这是考点，不是废话）
   2.4 几何级数与"主项支配"
   2.5 复杂度分析的工程细节
   2.6 经典反例：只看图会骗人
3. 第 2 章：分治 I —— Karatsuba、递推式与 Master 定理
   3.1 问题动机
   3.2 算法：Karatsuba
   3.3 复杂度分析
   3.4 Master 定理（本课最高频工具）
   3.5 经典反例：Master 定理用不了的三个递推
   3.6 经典反例：为什么必须数"所有操作"
4. 第 3 章：分治 II —— Strassen 矩阵乘法
   4.1 问题动机
   4.2 算法：Strassen 的 7 次乘法
   4.3 复杂度
   4.4 正确性
   4.5 经典反例与工程现实
5. 第 4 章：分治 III —— 选择问题（QuickSelect 与 Median-of-Medians）
   5.1 问题动机
   5.2 算法一：QuickSelect（随机枢轴）
   5.3 QuickSelect 的期望复杂度
   5.4 算法二：Median-of-Medians（确定性 $O(n)$）
   5.5 经典反例
6. 第 5 章：分治 IV —— 最近点对（Closest Pair）
   6.1 问题动机
   6.2 算法
   6.3 正确性：为什么只要查 7 个
   6.4 复杂度
   6.5 经典反例与陷阱
7. 第 6 章：图 —— BFS、DFS、拓扑序与强连通分量
   7.1 问题动机
   7.2 图的表示与复杂度约定
   7.3 BFS：无权最短路
   7.4 DFS 与拓扑排序
   7.5 强连通分量（Kosaraju 两遍 DFS）
   7.6 2-SAT：SCC 的经典应用
   7.7 经典反例汇总
8. 第 7 章：最短路径 —— Dijkstra、Bellman–Ford、Floyd–Warshall
   8.1 问题动机与三个变体
   8.2 Dijkstra
   8.3 Bellman–Ford
   8.4 Floyd–Warshall（APSP）
   8.5 选型速查
9. 第 8 章：贪心 I —— 区间调度与 Huffman 编码
   9.1 贪心的通用范式与"什么时候能贪"
   9.2 区间调度（Interval Scheduling）
   9.3 Huffman 编码
10. 第 9 章：贪心 II —— 最小生成树（MST）
   10.1 问题动机
   10.2 两个结构性定理（所有 MST 算法的基石）
   10.3 Kruskal：按边权从小到大加，不成环就加
   10.4 Prim：从一个点开始，每次加最轻的出边
11. 第 10 章：动态规划 I —— 从 Fibonacci 到编辑距离

- 11.1 问题动机：为什么需要 DP
- 11.2 加权区间调度（DP 的第一课）
- 11.3 最长递增子序列（LIS）
- 11.4 编辑距离（Edit Distance）
- 11.5 经典反例：DP 建模的三个常见错误
- 12. 第 11 章：动态规划 II —— 背包、链矩阵乘、TSP、树上独立集
  - 12.1 0/1 背包（Knapsack）
  - 12.2 链矩阵乘法（Chain Matrix Multiplication）
  - 12.3 旅行商问题（TSP）的 Held–Karp DP
  - 12.4 树上的最大独立集
- 13. 第 12 章：线性规划（Linear Programming）
  - 13.1 问题动机
  - 13.2 标准形式与几何
  - 13.3 单纯形法（Simplex）
  - 13.4 用 LP 表达经典问题
  - 13.5 整数规划（ILP）与松弛间隙
  - 13.6 LP 的建模陷阱
- 14. 第 13 章：最大流与最小割
  - 14.1 问题动机与定义
  - 14.2 为什么"贪心找路径"会错
  - 14.3 Ford–Fulkerson 算法
  - 14.4 最大流最小割定理
  - 14.5 二分匹配归约到最大流
  - 14.6 复杂度与经典反例
- 15. 第 14 章：对偶与零和博弈
  - 15.1 问题动机
  - 15.2 标准形与对偶的构造规则
  - 15.3 弱对偶与强对偶
  - 15.4 互补松弛（complementary slackness）
  - 15.5 例子：面包店的对偶
  - 15.6 零和博弈与 minimax
  - 15.7 经典反例
- 16. 第 15 章：P、NP 与 NP 完全性
  - 16.1 问题动机：为什么需要"难"的理论
  - 16.2 定义
  - 16.3 归约：方向决定一切
  - 16.4 Cook–Levin 定理与归约图谱
  - 16.5 四个必会归约的完整构造
  - 16.6 其它经典结果（知道结论与直觉即可）
  - 16.7 经典反例与常见错误
- 17. 第 16 章：应对 NP 难 —— 精确算法、FPT 与工程手段
  - 17.1 问题动机
  - 17.2 精确指数算法
  - 17.3 固定参数可解（FPT）
  - 17.4 近似算法与随机化（详见 §20、§21）
  - 17.5 工程实践：规约到 SAT / ILP / SMT
  - 17.6 经典反例
- 18. 专题一：证明方法论
  - 18.1 循环不变式（Loop Invariant）
  - 18.2 交换论证（Exchange Argument）
  - 18.3 摊还分析（Amortized Analysis）
  - 18.4 归纳法与强归纳
  - 18.5 反例证明（Disproof）
- 19. 专题二：归约的构造方法
  - 19.1 归约的四种常见形态
  - 19.2 构造归约的五个步骤
  - 19.3 gadget 设计的三个常用技巧
  - 19.4 归约速查表（完整版见速查文档）
  - 19.5 归约的三个经典错误
  - 19.6 判定版 vs 优化版
- 20. 专题三：近似算法
  - 20.1 定义与记号
  - 20.2 顶点覆盖的 2-近似
  - 20.3 度量 TSP 的 2-近似与 Christofides 1.5-近似
  - 20.4 集合覆盖的 $\ln n$ 近似
  - 20.5 近似算法速查
  - 20.6 经典反例
- 21. 专题四：随机算法
  - 21.1 为什么要随机
  - 21.2 随机快排（Randomized Quicksort）
  - 21.3 Karger 最小割（Karger's Contraction）
  - 21.4 哈希（Hashing）
- 22. 与已有笔记的衔接
  - 22.1 ← CS70：离散与概率
  - 22.2 ← CS229：凸优化与对偶
  - 22.3 ← 15-445：join 顺序与贪心/DP
- 23. 复习清单与自测题
  - 23.1 复习清单（按"必须能当场写出"分级）
  - 23.2 自测题（10 题，答案紧随）
- 24. 参考来源
  - 24.1 课程页面
  - 24.2 讲义 PDF（18 份，前缀 `https://cs170.org/assets/lec/`）
  - 24.3 作业与讨论课（抽样核验）
  - 24.4 教材（DPV 公开章节，作者主页）
  - 24.5 声明

1. 课程概览

1.1 这门课到底在教什么

Policies 页对课程目标的原话是：教学生"像算法学家一样思考"——**建模计算过程、掌握设计高效算法的核心概念、以及如何比较与衡量算法的进步**。它列出的覆盖范围是：图算法、贪心、分治、动态规划、线性规划、快速矩阵乘法、傅里叶变换、NP 完全性。

把这句官方描述翻译成可执行的学习目标，其实是三个问题（Lecture 1 与 Lecture 2 反复强调的同三个问题）：

1. **Does it work?**（正确吗）——需要**证明**，而不是"跑几个样例都对"。
2. **Is it fast?**（快吗）——需要**渐近分析**与**递推式求解**。
3. **Can I do better?**（还能更快吗）——需要知道**下界**、**问题的难度分类**（P / NP）、以及**放松目标后的近似与随机化**。

CS170 与多数"算法课"的区别在于第三问的权重。它花整整 5–6 次课在 NP 完全性与归约上，因为**识别一个问题是"难"的，和设计一个算法同等重要**：一旦确认 NP 难，工程上的正确动作就变成近似、随机化、或利用实例结构（分支限界、FPT、ILP/SAT 求解器），而不是继续找一个不存在的多项式算法。

1.2 先修与前置假设

先修	具体用到什么	在 CS170 中的出现位置
CS 61B（数据结构）	数组、链表、栈/队列、堆（优先队列）、哈希表、并查集、图的基本表示	堆 → Dijkstra / Huffman / Prim；并查集 → Kruskal；哈希 → 随机算法章
CS 70（离散数学与概率）	归纳法、反证法、图的定义、条件概率、期望、期望的线性性、Markov/Chebyshev 不等式	几乎所有正确性证明用归纳；随机算法章全程用期望与概率上界
微积分（非正式）	对数、指数、几何级数、积分给级数做界	渐近分析、递推式求解

官方先修就是 **CS 61B + CS 70**（Policies 页明写）。**没有**要求线性代数，但线性规划与对偶部分若有线性代数直觉会轻松很多（见 §8 与 CS229 的衔接）。

1.3 评分构成（Fall 2026，来自 Policies 页）

项目	占比	说明
In-Class Quizzes（随堂小测 ×3）	10%	Quiz 1 在 9/24，Quiz 2 在 10/29，Quiz 3 在 12/1（Policies 页列出，且注明"日期可能变动"）
Participation（到课率）	5%	全学期 4 次 lecture drop
Exams（两次期中 + 期末）	85%	Midterm 1 占 25%、Midterm 2 占 25%、Final 占 35%
Written Homework	0%	明确**不计分**，纯练习；每周日发布、下周六 22:00 截止

两个容易误判的点：

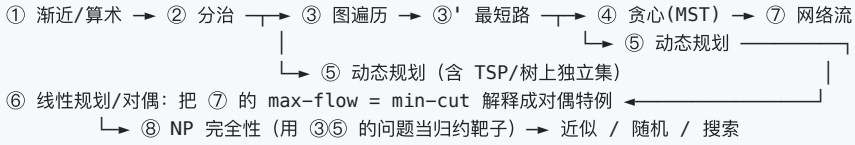
- **作业不计分但极其重要**。Lecture 2 的公告直说 "Class quizzes will be drawn from homeworks and discussions. So do the homework!"；Policies 页也把作业定位为"consolidate your understanding of the course material"的手段。作业是唯一带官方解答（`hw*-sol.pdf`）的训练材料，把它当"免费真题库"用。
- **评分构成与考试日期每个学期都可能变**。上表只对 Fall 2026 有效；若你在别的学期选课，**以课程当学期主页为准**。本笔记后面所有涉及"考什么"的判断都属于**推断**，请自行核对当学期的 calendar 与 policies。

1.4 八章讲义地图（Lecture 1–18 → 知识块）

课程主页把 18 次课按周排开；按知识依赖关系可以归并成 8 个块。这张表是后面所有章节的索引。**一处诚实说明**：Policies 页的覆盖范围还提到"傅里叶变换"（FFT），但 Fall 2026 公开的 18 份讲义里没有独立的 FFT 讲次，故本笔记不设 FFT 专章（**推断**：或并入分治选讲、或留作选学；以当学期课件为准）。

知识块	对应讲次（Fall 2026）	DPV 章节	核心问题	本笔记对应章
① 渐近分析与算术	Lec 1	§0.2, §0.3, §2.1–§2.2	怎么衡量"快"；整数乘法能不能比 $O(n^2)$ 快	第 1–2 章
② 分治	Lec 2–4	§2.3–§2.5	递推式求解（Master 定理）；矩阵乘法；选择；最近点对	第 2–5 章
③ 图：分解与路径	Lec 5–7	§3.1–§3.4, §4.1–§4.4, §4.6	BFS/DFS、拓扑序、强连通分量；最短路	第 6–7 章
④ 贪心	Lec 8–10	§5.1–§5.3	区间调度、Huffman 编码、最小生成树	第 8–9 章
⑤ 动态规划	Lec 11–14	§6	背包、编辑距离、LIS、TSP、树上独立集、链矩阵乘	第 10–11 章
⑥ 线性规划与对偶	Lec 15, 17	§7.1, §7.4	LP 的几何与单纯形；对偶；零和博弈	第 12、14 章
⑦ 网络流与匹配	Lec 16	§7.2	最大流最小割；二分匹配	第 13 章
⑧ 难解性与应对	Lec 18 + §9	§7.5, §8, §9	P/NP、归约、NP 完全；近似、随机、搜索	第 15–16 章 + §18–§21 专题

依赖关系（后面的块假设前面的块已掌握）：



一个实用的读法：**③ 和 ⑤ 是整门课的地基**。图遍历不会，MST 的 cut property 就只是背结论；DP 不会，§9 的近似算法里"树上的独立集"就直接卡住。

1.5 与 CS70 / MIT 6.006 的关系

与 CS70：CS70 给你"证明工具"和"概率工具"，CS170 给你"用它们分析算法"的场景。对应关系如下（不重复讲，只标接口）：

CS70 概念	在 CS170 的第一次真正使用
数学归纳法（强归纳）	第 2 章递推式求解；第 10 章 DP 正确性（对子问题 DAG 的拓扑序归纳）
反证法	第 1 章 $0.1n^2 \notin O(n)$；第 13 章最大流最小割
图论基础（度、路径、连通）	第 6 章 BFS/DFS、第 9 章割（cut）
期望的线性性	第 16 章随机快排的期望比较次数、Karger 最小割
条件期望 / 全期望公式	第 4 章 QuickSelect 期望运行时间
Markov / Chebyshev	第 7 章随机算法的失败概率放大（重复 $O(\log n)$ 次）

与 MIT 6.006：两门课主题高度重叠（分治、图、DP、最短路、最大流），差异在**重心**：

维度	CS170	6.006
证明	每章必证正确性，考试大量"证明或反例"	更偏实现与复杂度，证明要求较轻
难度理论	NP 完全性、归约占 ~1/4 篇幅，是核心	通常只占 1–2 次课
线性规划/对偶	有独立讲次，含零和博弈	一般不讲或只提一句
工程实现	伪代码层面，不要求写代码	有编程作业（Python）

所以如果你已经过了一遍 6.006，本课的新增量集中在：**归约的技巧**、**对偶**、以及**"证明"这件事本身的写法**（见 §18 证明方法论专题）。

2. 第 1 章：渐近分析与算法分析的写法

2.1 问题动机

同一段程序，在不同机器、不同编译器、不同输入上的运行时间可以差 10 倍以上。我们要一个**与机器无关**、又能**预测大规模行为**的度量。答案是"忽略常数、只看增长率"——但这句口号必须落成可写、可判、可反驳的定义，否则考试里"证明 $T(n) \in O(g(n))$"就无从下手。

2.2 定义

设 $T(n), g(n)$ 是正整数上的正函数。

$$T(n) \in O(g(n)) \iff \exists c>0,\ \exists n_0>0,\ \forall n \geq n_0:\ T(n) \leq c \cdot g(n)$$

$$T(n) \in \Omega(g(n)) \iff \exists c>0,\ \exists n_0>0,\ \forall n \geq n_0:\ c \cdot g(n) \leq T(n)$$

$$T(n) \in \Theta(g(n)) \iff T(n) \in O(g(n)) \text{ 且 } T(n) \in \Omega(g(n))$$

三个要点，考试里全是扣分点：

1. **量词顺序不可换**：c, n_0 只能依赖 g，不能依赖 n（否则任何函数都 $O(1)$）。
2. **"存在"意味着你只需给出一组可行的 (c, n_0)**，不需要最优的那组。
3. **要给出最紧且最简单的界**。$5n^2 \in O(n^3)$ 是真命题，但作为答案会被判"不够紧"。

2.3 证明模板（这是考点，不是废话）

证 $T \in O(g)$（构造法）：写出不等式 $T(n) \leq c\,g(n)$，把 T 里增长最快的项放大到 g，解出成立的 n 范围，取 c 与 n_0。

证 $T \notin O(g)$（反证法）：假设存在 c, n_0 使 $\forall n \geq n_0$ 有 $T(n) \leq c\,g(n)$；然后**构造一个依赖 c, n_0 的 n** 使不等式破裂。

讲义的标准例子：

- **正例**：$T(n) = 2n^2 + 2$，取 $c = 4, n_0 = 1$，则 $2n^2 + 2 \leq 4n^2$ 对 $n \geq 1$ 成立，故 $T \in O(n^2)$。
- **反例**：证 $0.1n^2 \notin O(n)$。假设有 c, n_0 使 $\forall n \geq n_0$：$0.1n^2 \leq cn$，两边除以 n 得 $0.1n \leq c$。取 $n = n_0 + 10c$，则 $n \geq n_0$ 但 $0.1n = 0.1n_0 + c > c$，矛盾。□

注意反证里 n **同时依赖** n_0 与 c——这正是"量词顺序"在证明中的体现。

2.4 几何级数与"主项支配"

讲义要求会证：对常数 r，

$$T(n) = 1 + r + r^2 + \cdots + r^n \in \begin{cases} \Theta(r^n) & r > 1 \\ \Theta(1) & r < 1 \\ \Theta(n) & r = 1 \end{cases}$$

$r \neq 1$ 时用等比求和 $T(n) = \frac{r^{n+1}-1}{r-1}$；$r > 1$ 时 $T(n) \leq \frac{r}{r-1} r^n$（上界）且 $T(n) \geq r^n$（下界），故 $\Theta(r^n)$。这条结论在递归树分析里反复用到：**当每层工作量按常数因子衰减时，总工作量由根节点决定；按常数因子增长时，由叶节点决定。**

2.5 复杂度分析的工程细节

场景	该写什么	常见错误
顺序执行两段	相加，取大者	写成相乘
嵌套循环	相乘	内层依赖外层上界时忘记求和
`for i in 1..n: for j in i..n:`	$\sum_i (n-i+1) = \Theta(n^2)$	直接写 n^2 却不说明
递归	写递推式再解（Master 定理，见第 2 章）	漏掉"每层的非递归开销"
图算法	用 $n=	V

图算法的复杂度必须用 n 和 m 两个变量。这是 CS170 的硬性习惯：$O(n+m)$ 与 $O(n^2)$ 在稀疏图上差一个量级，而"稀疏图"恰恰是绝大多数实际输入。

2.6 经典反例：只看图会骗人

讲义专门用一页幻灯片讲这个陷阱：画图时 $T(n) = 0.1n^2 + 2$ 与 $g(n) = n\log n$ 在 n 较小时看起来"T 在 g 下面"，于是有人断言 $T \in O(n\log n)$。错。因为 $n\log n / n^2 = \log n / n \to 0$，任何常数 c 都撑不住大 n。

教训：渐近命题必须用 c, n_0 的代数证明，不能用"我画了一下"。

3. 第 2 章：分治 I —— Karatsuba、递推式与 Master 定理

3.1 问题动机

两个 n 位十进制（或 n 比特二进制）整数相乘，小学竖式是 $O(n^2)$：n 个部分积，每个 n 位。1960 年 Karatsuba 发现可以做到 $O(n^{1.585})$。这不是玩具问题：大整数乘法是 RSA、精确计算、π 的位数计算的底层原语；今天最快的算法（Harvey–van der Hoeven 2019）达到 $O(n\log n)$，讲义明确列出这条演进链：

算法	年份	复杂度
竖式	—	$O(n^2)$
Karatsuba	1960	$O(n^{\log_2 3}) \approx O(n^{1.585})$
Toom–3 / Toom–Cook	1963	$O(n^{1.465})$
Schönhage–Strassen	1971	$O(n\log n\log\log n)$
Fürer	2007	$O(n\log n \cdot 2^{O(\log^* n)})$
Harvey–van der Hoeven	2019	$O(n\log n)$

3.2 算法：Karatsuba

把 x, y 各拆成高低两半（以 $B = 10^{n/2}$ 为基）：

$$x = aB + b, \qquad y = cB + d$$

朴素展开需要 4 次 $n/2$ 位乘法：$ac,\ ad,\ cb,\ bd$。Karatsuba 的关键恒等式（讲义标注"Gauss 当年对复数乘法也用过这招"）：

$$ad + cb = (a+b)(c+d) - ac - bd$$

于是只需 3 次 $n/2$ 位乘法：

```
Karatsuba(x, y, n):            # x, y 为 n 位非负整数，n 为 2 的幂
    if n == 1: return x * y
    B = 10^(n/2)               # 二进制时 B = 2^(n/2)
    a, b = floor(x / B), x mod B
    c, d = floor(y / B), y mod B
    Q1 = Karatsuba(a, c, n/2)          # ac
    Q2 = Karatsuba(b, d, n/2)          # bd
    Q3 = Karatsuba(a + b, c + d, n/2)  # (a+b)(c+d)
    return Q1 * B^2 + (Q3 - Q1 - Q2) * B + Q2
```

注意 $a+b$ 可能进位到 $n/2+1$ 位。工程上要么用 $n/2+1$ 位的递归，要么先判断进位再修正。这是本算法最容易被忽略的实现细节（**推断**：讲义在 "Details we skipped" 一页提到加减法归入低阶项，未展开此细节）。

3.3 复杂度分析

设 $T(n)$ 为位数 n 的乘法代价。每次递归 3 个子问题，规模减半，本层加减与移位共 $O(n)$（讲义给出更紧的常数上界 $20n$）：

$$T(n) = 3T(n/2) + O(n), \qquad T(1) = O(1)$$

用递归树：第 t 层有 3^t 个节点，每个规模 $n/2^t$，本层总开销 $3^t \cdot O(n/2^t) = O(n)\,(3/2)^t$。层数 $\log_2 n$。几何级数公比 $3/2 > 1$，由 §2.4 的结论，总和由**最后一层**支配：

$$T(n) = O\big(n \cdot (3/2)^{\log_2 n}\big) = O\Big(n \cdot n^{\log_2(3/2)}\Big) = O\big(n^{\log_2 3}\big) \approx O(n^{1.585})$$

叶节点数是 $3^{\log_2 n} = n^{\log_2 3}$，这也直接给出答案：**叶子数 × 每叶代价 $O(1)$ 即总代价的主项**。

3.4 Master 定理（本课最高频工具）

> **定理（Master Theorem）** 设 $a \geq 1,\ b > 1,\ d \geq 0$ 为与 n 无关的常数，$T(n) = aT(n/b) + O(n^d)$。则
>
> $$T(n) \in \begin{cases} O(n^d) & a < b^d \\ O(n^d \log n) & a = b^d \\ O(n^{\log_b a}) & a > b^d \end{cases}$$
>
> 且这三种情形下的界都是紧的（Θ）。

直觉（务必内化，考试常考"为什么"）：比较 a 与 b^d，就是比较"每层工作量是涨还是缩"。

```
a > b^d：宽树                  a < b^d：窄高树              a = b^d：平衡
根 ──────────── 1              根 ──────────── n^d          每层都是 θ(n^d)
│                              │
├─ a 个 ──── 每层 ×(a/b^d)      ├─ a 个 ──── 每层 ×(a/b^d)<1
▼                              ▼
叶子最多 → θ(n^{log_b a})      根最多 → θ(n^d)              共 log_b n 层 → θ(n^d log n)
```

使用前提（三个"必须检查"）：

1. 子问题规模必须**严格按同一因子 b 缩小**（n/b、$\lceil n/b \rceil$、$n/b + O(1)$ 都可以，差常数不影响）。
2. 本层开销必须是**多项式 $O(n^d)$**，不能是 $n\log n$、$n/\log n$、2^n。
3. a, b, d 必须是**常数**，不能随 n 变。

3.5 经典反例：Master 定理用不了的三个递推

这三个例子是 CS170 的高频"陷阱题"，务必能当场手解。

反例 1：$T(n) = 2T(n/2) + n\log n$。 形式上是 $a = 2, b = 2, d = 1$，$a = b^d$，套定理会得到 $\Theta(n\log n)$——**这是错的**。正确做法是递归树逐层求和：第 t 层有 2^t 个节点，每个规模 $n/2^t$，本层开销 $2^t \cdot \frac{n}{2^t}\log\frac{n}{2^t} = n(\log n - t)$。求和：

$$T(n) = \sum_{t=0}^{\log n} n(\log n - t) = n\sum_{k=0}^{\log n} k = \Theta(n\log^2 n)$$

错因：本层开销 $n\log n$ 不是 $O(n^d)$ 形式（$d = 1$ 时 $O(n)$ 装不下它）。**Master 定理的 $O(n^d)$ 项是硬约束，超了就不能用。**

反例 2：$T(n) = 2T(\sqrt{n}) + \log n$。 这里子问题规模是 $\sqrt{n}$ 而非 n/b，b 随 n 变，定理不适用。换元 $n = 2^k$，$S(k) = T(2^k)$：

$$S(k) = 2S(k/2) + k \implies S(k) = \Theta(k\log k) \implies T(n) = \Theta(\log n \log\log n)$$

反例 3：$T(n) = T(n-1) + n$。 规模只减不除，属于"减治"（decrease-and-conquer），Master 定理不适用。展开得 $T(n) = \sum_{k=1}^{n} k = \Theta(n^2)$。讲义给了一个漂亮的上下界技巧：上界"每项 $\leq n$ 所以 $\leq n^2$"，下界"只取后 $n/2$ 项，每项 $\geq n/2$，所以 $\geq n^2/4$"——**下界靠"砍掉一半项"**，这是估算和式的通用手法。

3.6 经典反例：为什么必须数"所有操作"

初学者常写"$T(n) = 3T(n/2)$ 所以 $O(n^{1.585})$"，漏掉 $+O(n)$。在这个例子里漏掉不影响结论（因为 $a > b^d$，叶子支配），但换成 $a = 2, b = 2, d = 1$ 就会把 $\Theta(n\log n)$ 错写成 $\Theta(n)$。**规则：先写完整的递推式，再决定哪一项可以忽略。**

4. 第 3 章：分治 II —— Strassen 矩阵乘法

4.1 问题动机

$n \times n$ 矩阵乘法是数值计算、图形学、以及（讲义特意点出的）Transformer 注意力机制的核心算子。朴素算法 $O(n^3)$：n^2 个输出元素，每个是长度 n 的点积。

分块递归：把 X, Y 各切成 4 块 $n/2 \times n/2$，

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} E & F \\ G & H \end{pmatrix} = \begin{pmatrix} AE+BG & AF+BH \\ CE+DG & CF+DH \end{pmatrix}$$

需要 8 次 子矩阵乘法，递推 $T(n) = 8T(n/2) + O(n^2)$。由 Master 定理 $a = 8, b^d = 4$，$a > b^d$，得 $T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$ ——和朴素算法同阶，白折腾。

4.2 算法：Strassen 的 7 次乘法

Strassen（1969）用 7 次乘法 + 若干次加减法算出同样的 4 个块。一种常见的写法（**推断**：讲义只给"8 减到 7"的思路与结论，具体 7 个乘积的公式各教材写法略有差异，此处采用 DPV/标准教材版本）：

```
M1 = (A + D)(E + H)
M2 = (C + D) E
M3 = A (F − H)
M4 = D (G − E)
M5 = (A + B) H
M6 = (C − A)(E + F)
M7 = (B − D)(G + H)

左上 = M1 + M4 − M5 + M7      # AE + BG
右上 = M3 + M5                # AF + BH
左下 = M2 + M4                # CE + DG
右下 = M1 − M2 + M3 + M6      # CF + DH
```

4.3 复杂度

$$T(n) = 7T(n/2) + O(n^2) \implies a = 7,\ b^d = 2^2 = 4,\ a > b^d \implies T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

4.4 正确性

把 7 个 M_i 展开代入即可验证，例如右下角：

$$M_1 - M_2 + M_3 + M_6 = (AE + AH + DE + DH) - (CE + DE) + (AF - AH) + (CE + CF - AE - AF) = CF + DH \checkmark$$

其余三块同理。**证明的本质是"代数恒等式验证"，不需要归纳**——这与 Karatsuba 一样：分治算法的正确性往往只是初等代数，难点在"怎么想到这个恒等式"。

4.5 经典反例与工程现实

- **渐近更优 ≠ 实际更快**。Strassen 的常数因子大、递归到小矩阵时 cache 行为差、且数值稳定性不如朴素算法（加法增多会放大舍入误差）。实践中 BLAS 用高度优化的 $O(n^3)$ 分块算法，只在 n 极大且精度要求可控时才考虑 Strassen 类算法。（**推断**：讲义未讨论数值稳定性，这是数值线性代数的常识。）
- **不能无限递归**。递归到 $n \le$ 阈值（如 64）就切换成朴素算法，否则加减法开销会吃掉收益。
- **矩形矩阵**要先 padding 成方阵，这会浪费空间；实际库用 Strassen–Winograd 变体处理非方情形。

5. 第 4 章：分治 III —— 选择问题（QuickSelect 与 Median-of-Medians）

5.1 问题动机

k-select 问题：给定含 n 个数的数组 S 与 $k \in \{1, \dots, n\}$，求第 k 小的元素。$k=1$ 是最小值，$k=n$ 是最大值，$k=n/2$ 是中位数。排序后取第 k 个是 $O(n \log n)$。问题是：**能不能只花 $O(n)$？** 直觉上可以——求最小值显然只要 $O(n)$，中位数"信息量"不该比最小值贵一个 $\log$ 因子。这一章给两个答案：随机化的 QuickSelect（期望 $O(n)$）与确定性的 Median-of-Medians（最坏 $O(n)$）。

5.2 算法一：QuickSelect（随机枢轴）

```
QuickSelect(S, k):
    if |S| == 1: return S[0]
    从 S 中均匀随机选一个元素 v 作为枢轴(pivot)
    S_L = [x in S : x < v]
    S_v = [x in S : x == v]
    S_R = [x in S : x > v]
    if k <= |S_L|:                return QuickSelect(S_L, k)
    elif k <= |S_L| + |S_v|:      return v
    else:                         return QuickSelect(S_R, k - |S_L| - |S_v|)
```

5.3 QuickSelect 的期望复杂度

核心概念：好枢轴（good pivot）。若枢轴落在"第 $n/4$ 小到第 $3n/4$ 小"之间，则划分后两侧各不超过 $3n/4$，称其为好枢轴。

引理：至少一半元素是好枢轴。 *证明*：小于第 $n/4$ 小的元素至多 $n/4$ 个，大于第 $3n/4$ 小的元素至多 $n/4$ 个，故落在 $[n/4, 3n/4]$ 区间内的元素至少 $n/2$ 个。□

因此**均匀随机选枢轴，命中好枢轴的概率 $\geq 1/2$**。

定理：QuickSelect 的期望运行时间 $E[T(n)] = O(n)$。

证明：把随机性对"是否好枢轴"做一次粗糙的分情况（这是讲义采用的写法）：

$$E[T(n)] \leq \underbrace{O(n)}_{\text{划分}} + \underbrace{\tfrac{1}{2}\,T(3n/4)}_{\text{好枢轴}} + \underbrace{\tfrac{1}{2}\,T(n)}_{\text{坏枢轴（规模至多 } n-1\text{）}}$$

移项：$\frac{1}{2}E[T(n)] \leq O(n) + \frac{1}{2}T(3n/4)$，即

$$E[T(n)] \leq T(3n/4) + O(n)$$

（常数被吸收进 $O(n)$。）由 Master 定理：$a=1,\ b=4/3,\ d=1$，$a = 1 < (4/3)^1 = b^d$，属于"根支配"情形，$E[T(n)] = O(n)$。用递归树看更直观：第 i 层规模 $\leq n(3/4)^i$，本层开销 $cn(3/4)^i$，等比求和

$$E[T(n)] \leq \sum_{i=0}^{\log_{4/3} n} cn\left(\tfrac{3}{4}\right)^i \leq \frac{cn}{1 - 3/4} = 4cn = O(n)$$

□

一个必须澄清的常见错误：有人用"指示器变量 + 全部元素对"来算 QuickSelect 的期望比较次数。设排序后元素 $z_1 < \cdots < z_n$，事件"z_i 与 z_j 被比较"在二者同处一个子问题时概率为 $\frac{2}{j-i+1}$，于是求和

$$\sum_{i<j} \frac{2}{j-i+1} = \sum_{d=1}^{n-1} \frac{2(n-d)}{d+1} = (n+1)H_n - 2n = \Theta(n \log n)$$

这个式子**对随机快排（Quicksort）是紧的**（期望比较次数 $\approx 2n \ln n$），但**对 QuickSelect 只是松上界**：QuickSelect 每层只递归进入**一侧**，绝大多数 (i, j) 对所在的子问题根本不会被访问。想要 $O(n)$ 的紧界，必须用上面"好枢轴 + 只走一侧"的递归式，不能直接套快排的求和。**这是 CS170 里"公式对但用错场景"的典型案例。**

5.4 算法二：Median-of-Medians（确定性 $O(n)$）

随机化给出的是**期望** $O(n)$，最坏仍是 $O(n^2)$（例如输入已排序且枢轴固定取首元素，每次划分都极不平衡）。要**最坏** $O(n)$，必须确定性地选一个好枢轴。思路：花 $O(n)$ 时间**造**一个好枢轴。

```
MedianOfMedians(S):                 # 返回一个"足够好"的枢轴
    把 S 任意分成 ceil(n/5) 组，每组 5 个（最后一组可能不足）
    对每组做常数时间的排序（5 个元素最多 7 次比较），取该组中位数
    令 M 为这 ceil(n/5) 个中位数组成的集合
    return Select(M, |M|/2)         # 递归求 M 的中位数
```

关键引理：这样选出的枢轴 v 满足 $\geq 3n/10$ 个元素 $\leq v$，且 $\geq 3n/10$ 个元素 $\geq v$。

证明：设有 $g = \lceil n/5 \rceil$ 组。中位数**大于** v 的组至少有 $g/2$ 组（因为 v 是这些中位数的中位数）。在这些组里，每组有 3 个元素 $\geq$ 该组中位数 $\geq v$，故 $\geq 3 \cdot \frac{g}{2} \geq \frac{3n}{10}$ 个元素 $\geq v$。对称地另一半。□

于是递归式：

$$T(n) \leq \underbrace{T(n/5)}_{\text{找中位数的中位数}} + \underbrace{T(7n/10)}_{\text{递归进入较大一侧}} + O(n)$$

检查是否收敛：$1/5 + 7/10 = 9/10 < 1$。用"猜+验"：设 $T(n) \leq cn$，则

$$T(n) \leq \frac{cn}{5} + \frac{7cn}{10} + an = \frac{9cn}{10} + an \leq cn \iff an \leq \frac{cn}{10} \iff c \geq 10a$$

取 $c = 10a$ 即可，基础情况取足够大的 n_0。故 $T(n) = O(n)$。**注意这里 Master 定理不适用**（两个不同规模的子问题），必须用代换法或"总规模按几何级数收缩"的论证。

常数很大：分组大小 5 时 $c \geq 10a$，实际常数约 20–30 倍。这是"理论优美、工程不用"的典型——实践中 QuickSelect（随机枢轴 + 三数取中）更快。

5.5 经典反例

反例	现象	结论
QuickSelect + 固定取首元素枢轴，输入已升序	每次划分出 0 与 $n-1$，$T(n) = T(n-1) + O(n) = \Theta(n^2)$	随机化不是装饰，是正确性/复杂度的一部分
QuickSelect + "三数取中"	存在针对性构造的杀手输入使其 $O(n^2)$（**推断**：需专门构造，讲义未给）	确定性启发式没有最坏保证
Median-of-Medians 分组取 3	$T(n) \leq T(n/3) + T(2n/3) + O(n)$，$1/3 + 2/3 = 1$，代换法失效（只能得到 $O(n \log n)$）	**分组大小 5 是"能收敛的最小奇数"**，这是本算法最精妙之处
试图用排序解决 select	$O(n \log n)$	比较模型下 select 的下界是 $\Omega(n)$，排序不是最优

> **分组 3 为什么失败**：$T(n) \leq T(n/3) + T(2n/3) + O(n)$ 恰好卡在临界点，递归树每层开销都是 $\Theta(n)$、层数 $\Theta(\log n)$，总计 $\Theta(n \log n)$。分组 5 时两侧比例和 $9/10 < 1$，才有几何衰减。

6. 第 5 章：分治 IV —— 最近点对（Closest Pair）

6.1 问题动机

平面上 n 个点，求距离最近的一对。暴力 $O(n^2)$（$\binom{n}{2}$ 对）。这个问题是计算几何的"hello world"，也是分治思想的试金石：**难的不是分，而是 merge 步骤怎么做到线性**。应用包括碰撞检测、聚类初始化、图形学中的最近邻查询。

6.2 算法

```
ClosestPair(P):                          # P 按 x 坐标排序
    if |P| <= 3: 暴力返回最近点对
    用 x 中位数把 P 分成左半 L 与右半 R
    (pL, dL) = ClosestPair(L)
    (pR, dR) = ClosestPair(R)
    d = min(dL, dR);  (p, q) = 对应的那一对
    考虑"跨中线"的点对：取所有 |x − x_mid| < d 的点，按 y 排序得 S
    for i = 1..|S|:
        for j = i+1 .. min(i+7, |S|):     # 只查后面 7 个!
            if dist(S[i], S[j]) < d: d = dist(...); (p,q) = (S[i],S[j])
    return (p, q)
```

两个非平凡的设计：

1. **只检查 $|x - x_{mid}| < d$ 的条带**（strip）。距离 $< d$ 的跨中线点对，两点都必须落在条带内。
2. **条带内按 y 排序后，每个点只需与后续 7 个点比较**。

6.3 正确性：为什么只要查 7 个

引理：设条带内点按 y 升序为 $s_1, \ldots, s_k$。若 $j > i + 7$，则 $d(s_i, s_j) \geq d$。

证明：把条带（宽 $2d$）按 $d/2 \times d/2$ 的方格切分。考虑以 s_i 为中心、边长为 $2d$ 的矩形（y 方向从 $s_i.y$ 到 $s_i.y + 2d$，x 方向宽 $2d$）。这个矩形可被 $4 \times 4 = 16$ 个 $d/2$ 方格覆盖；而**每个 $d/2$ 方格内至多一个点**——因为若某方格内有两个点，它们距离 $\leq \frac{d}{2}\sqrt{2} < d$，与"$d$ 是左右两侧各自的最小距离"矛盾（该方格完全落在左半或右半，取决于中线位置；跨中线的方格需要更细的论证，标准做法是把条带按 x 中线再分成两半，每半宽 d，各用 2×4 个方格覆盖）。

在 y 方向跨度 $2d$ 内、每格至多一点，共 $O(1)$ 个点。精确计数给出**至多 7 个**（讲义给的常数；不同教材给 7 或 15，$O(1)$ 即可）。因此 $j > i + 7$ 时 y 差已 $\geq d$，距离必 $\geq d$。□

> 严谨版：把条带分成左条带（$x \in [x_{mid} - d, x_{mid}]$）与右条带（$x \in [x_{mid}, x_{mid} + d]$）。在每个半条带内，按 $d/2$ 方格切分，y 跨度 $2d$ 只需 $4 \times 2 = 8$ 格，每格至多一点。一个点至多需要检查同一半条带的 7 个邻居 + 另一半条带的常数个。**常数是 7 还是 15 不影响 $O(n \log n)$**，考试写清"常数个"即可。

6.4 复杂度

$$T(n) = 2T(n/2) + O(n) \implies a = 2,\ b^d = 2,\ a = b^d \implies T(n) = \Theta(n \log n)$$

其中 $O(n)$ merge 的前提是"按 y 排序"能在 $O(n)$ 完成——做法是**在递归返回时归并两个已按 y 排序的子列表**（类似归并排序），而不是每层重新排序（否则变成 $O(n \log^2 n)$）。

6.5 经典反例与陷阱

陷阱	后果	正确做法
每层对条带重新排序	$T(n) = 2T(n/2) + O(n \log n) \to O(n \log^2 n)$	递归中维护 y 有序，归并
条带内检查所有点对	merge 变 $O(k^2)$，最坏 $O(n^2)$	用"7 邻居"引理
用 x 均值而非中位数分割	可能 $1 : n-1$ 分割，退化为 $O(n^2)$	必须用中位数保证平衡
认为"最近点对必在相邻 x 序"	反例：三点构成窄高三角形，最近对在 x 序上不相邻	必须查条带

7. 第 6 章：图 —— BFS、DFS、拓扑序与强连通分量

7.1 问题动机

图是 CS170 后半程的通用语言：最短路、MST、最大流、归约（NP 完全性里的"图问题"）全部建立在遍历之上。本章解决四个"结构性问题"：

1. 无权图上的**最短路径**（BFS）；

2. 有向无环图上的**拓扑序**；
3. 有向图的**强连通分量**（SCC）；
4. 无向图的**连通分量**与二分性检测。

7.2 图的表示与复杂度约定

表示	空间	判断 $(u,v)\in E$	遍历邻居
邻接矩阵	$O(n^2)$	$O(1)$	$O(n)$
邻接表	$O(n+m)$	$O(\deg u)$	$O(\deg u)$

约定：本笔记一律用 $n=|V|,\ m=|E|$，邻接表存储，因此 BFS/DFS 都是 $O(n+m)$。用邻接矩阵的算法（如 Floyd–Warshall）会显式标注 $O(n^3)$。

7.3 BFS：无权最短路

```
BFS(G, s):
    for all v: dist[v] = INF, parent[v] = NIL
    dist[s] = 0; Q = queue([s])
    while Q not empty:
        u = Q.dequeue()
        for v in Adj[u]:
            if dist[v] == INF:
                dist[v] = dist[u] + 1
                parent[v] = u
                Q.enqueue(v)
    return dist, parent
```

正确性（循环不变式 + 分层论证）：

> **不变式**：BFS 队列中的元素满足 `dist` 值单调不减，且相邻元素之差 ≤ 1；出队时 `dist[u]` 已是最短距离。

证明：归纳。初始队列只有 s，`dist=0`。假设当前出队 u，其 `dist` 正确。对每个未访问邻居 v，第一条到达 v 的路径必经过某个 `dist` 最小的前驱，故 `dist[v]=dist[u]+1` 是最短的（任何更短路径的前驱会更早出队）。队列单调性来自"新入队的 `dist` 只可能是当前层或下一层"。□

性质：BFS 树中 $s\to v$ 的路径就是一条最短路径（无权）。BFS 也能在 $O(n+m)$ 判断二分图（按层奇偶染色）。

经典反例：BFS 在**带权**图上不给最短路。反例：$s-a$ 权 1，$s-b$ 权 100，$b-a$ 权 1。BFS 得 $dist(a)=1$（走 $s\to a$），而真正最短路是 $s\to b\to a$ 权 101? 不，101>1。换一个：$s-a$ 权 100，$s-b$ 权 1，$b-a$ 权 1。BFS 给 $dist(a)=1$（边数），但带权最短路是 $s\to b\to a=2$。**边数少 ≠ 权重小**。

7.4 DFS 与拓扑排序

```
DFS(G):
    for all v: visited[v] = False
    timer = 0
    for all v: if not visited[v]: DFS-Visit(v)

DFS-Visit(u):
    visited[u] = True; disc[u] = ++timer
    for v in Adj[u]:
        if not visited[v]: parent[v] = u; DFS-Visit(v)
    fin[u] = ++timer
    # 后序：把 u 压入栈 / 追加到列表
```

拓扑排序（DAG 上）：反复取入度为 0 的点（Kahn 算法），或 DFS 后按**完成时间递减**输出。

定理：G 有拓扑序 $\iff$ G 是 DAG。

证明：（⇐）DAG 必有入度 0 的点（否则沿入边回溯必成环）；删去它仍是 DAG，归纳。（⇒）若有环 $v_1 \to \cdots \to v_k \to v_1$，拓扑序要求 v_1 在 v_2 前、…、v_k 在 v_1 前，矛盾。□

DFS 完成时间的核心性质：对任意边 (u, v)，
- 若 v 是 u 的后代（树边/前向边）：$[disc(u), fin(u)]$ 包含 $[disc(v), fin(v)]$；
- 若 v 是 u 的祖先（回边 back edge）：$[disc(v), fin(v)]$ 包含 $[disc(u), fin(u)]$；
- 否则两个区间**不相交**（横跨边 cross edge）。

这条"区间要么嵌套要么不相交"的性质是 SCC 算法与 2-SAT 的基础。**它也是判断有向图是否有环的依据：DFS 中出现回边 $\iff$ 有环。**

7.5 强连通分量（Kosaraju 两遍 DFS）

定义：u, v 强连通指 $u \rightsquigarrow v$ 且 $v \rightsquigarrow u$。强连通关系是等价关系，其等价类即 SCC。把每个 SCC 缩成一个点得到**condensation graph**，它一定是 DAG。

```
Kosaraju(G):
    1. 在 G 上跑 DFS，记录每个点的完成时间 fin[]
    2. 构造反向图 G^R（所有边反向）
    3. 按 fin[] 从大到小遍历顶点，在 G^R 上做 DFS；
       每一棵 DFS 树就是一个 SCC
```

正确性直觉：设 C 是完成时间最晚的 SCC（在 G 的 DFS 中）。C 在 condensation DAG 中是"源"（无入边来自未访问的 SCC），因此在 G^R 中它是"汇"，从 C 出发在 G^R 上 DFS 不会漏到别的 SCC。删去 C 后归纳。

复杂度：两遍 DFS，$O(n + m)$。

经典反例：只跑一遍 DFS 并按发现顺序输出，**不能**得到 SCC。反例：$a \to b$，$b \to a$，$b \to c$。从 a 出发的 DFS 会一路访问到 c，把 $\{a, b, c\}$ 误判为一个 SCC。**必须用完成时间 + 反向图。**

7.6 2-SAT：SCC 的经典应用

问题：给 n 个布尔变量与若干子句，每个子句形如 $(x_i \vee x_j)$（两个字面量），问是否存在满足赋值。

归约：每个文字 ℓ 建一个点。子句 $(\ell_1 \vee \ell_2)$ 等价于两个蕴含 $\neg\ell_1 \to \ell_2$ 与 $\neg\ell_2 \to \ell_1$，建成两条有向边。

定理：2-SAT 实例可满足 $\iff$ 对每个变量 x，x 与 $\neg x$ **不在同一个 SCC** 中。

证明：（⇒）若 $x, \neg x$ 在同一 SCC，则存在 $x \rightsquigarrow \neg x$ 与 $\neg x \rightsquigarrow x$ 的路径，即 $x \Rightarrow \neg x$ 且 $\neg x \Rightarrow x$，无论 x 取真取假都矛盾。（⇐）若不存在这样的 SCC，可按 condensation DAG 的拓扑逆序给每个 SCC 赋"真/假"（先处理"汇"SCC），得到的赋值满足所有蕴含；详见 DPV §3.3 的构造。□

这条定理的意义：2-SAT 在 P 里（$O(n + m)$），而 3-SAT 是 NP 完全的（第 15 章）。**每个子句多一个文字，问题从"线性时间"跳到"NP 完全"**——这是全课程最震撼的对比之一。

7.7 经典反例汇总

反例	现象
BFS 用于带权图	返回"边数最少"而非"权重最小"
DFS 后序直接当拓扑序（不反转）	得到的是逆拓扑序
拓扑排序用于有环图	必有剩余点无法输出，应报错而非强行输出
单遍 DFS 求 SCC	见 §7.5 反例
用 DFS 求无权最短路	DFS 找到的路径可能很长（取决于邻居顺序）

8. 第 7 章：最短路径 —— Dijkstra、Bellman–Ford、Floyd–Warshall

8.1 问题动机与三个变体

变体	输入	输出	主力算法
单源最短路（SSSP）	图 + 源 s，非负权	所有 $dist(s, v)$	Dijkstra $O((n + m) \log n)$
单源最短路，允许负权	图 + s，可能有负边	最短路或报"负环"	Bellman–Ford $O(nm)$

变体	输入	输出	主力算法
全对最短路（APSP）	图，可能有负边	所有 $dist(u,v)$	Floyd–Warshall $O(n^3)$
DAG 上的 SSSP	DAG + s	所有 $dist$	拓扑序 DP，$O(n+m)$

为什么 DAG 上是线性的：DAG 没有环，按拓扑序处理每个点时它的所有前驱都已算完，一次松弛即可，不需要"反复迭代"。这是"用结构换速度"的范例。

```
DAG-SSSP(G, s):
    按拓扑序排列顶点（第 6 章）
    dist[s] = 0, 其余 = INF
    for u in 拓扑序:
        for (u,v,w) in Adj[u]:
            if dist[u] + w < dist[v]: dist[v] = dist[u] + w
```

8.2 Dijkstra

前提：所有边权 $w_e \geq 0$。

```
Dijkstra(G, s):
    for all v: dist[v] = INF, prev[v] = NIL
    dist[s] = 0
    H = 最小堆，包含 (0, s)；visited = {}
    while H 非空:
        d, u = H.extractMin()
        if u in visited: continue
        visited.add(u)
        for (u, v, w) in Adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w; prev[v] = u
                H.decreaseKey(v, dist[v])   # 或直接 push (dist[v], v)
    return dist, prev
```

正确性（贪心 + 归纳，本质是"标签设定"论证）：

> **不变式**：每次从堆中取出 u 时，`dist[u]` 等于真正的最短距离。

证明：设 u 是第一个被取出但 `dist[u]` 不等于真值 $d^*(s,u)$ 的点。考虑真正最短路 $s \to \cdots \to x \to y \to \cdots \to u$，其中 x 是"已确定"的最后一个点，y 是第一个"未确定"的点。因为 x 已确定且边 (x,y) 被松弛过，`dist[y] <= dist[x] + w(x,y) = d^*(s,y)`。又因边权非负，$d^*(s,y) \leq d^*(s,u)$。所以 `dist[y] <= d^*(s,u) <= dist[u]`（后者因为 u 先于一切未确定点被取出，而 `dist[u]` 是堆中最小）。另一方面 `dist[u]` 不可能小于真值（任何 `dist` 值都对应一条真实路径）。故 `dist[u] = d^*(s,u)`，矛盾。□

复杂度：

实现	时间	适用
数组找最小	$O(n^2)$	稠密图 $m \approx n^2$
二叉堆	$O((n+m)\log n)$	通用
斐波那契堆	$O(m + n\log n)$	理论最优（实践中常数大）

经典反例（必考）：Dijkstra 在负权边上下错。

```
      s --2--> a
      s --5--> b
      b --(-4)--> a
```

Dijkstra 先取出 a（`dist=2`），标记确定，之后不再更新；b 取出后松弛 $b \to a$ 得 $5 + (-4) = 1$，但 a 已确定、不再更新。结果 `dist[a]=2`，而真值是 1。**错因**：正确性证明里用了"边权非负 $\Rightarrow d^*(s,y) \leq d^*(s,u)$"，负边直接摧毁这一步。

8.3 Bellman–Ford

思想：最短路至多 $n-1$ 条边；做 $n-1$ 轮"对所有边松弛"，第 i 轮后保证所有"至多 i 条边"的最短路已正确。

```
BellmanFord(G, s):
    dist[s] = 0, 其余 = INF
    repeat n-1 times:
        for each edge (u, v, w):
            if dist[u] + w < dist[v]: dist[v] = dist[u] + w
    # 第 n 轮：若还能松弛，说明存在负环
    for each edge (u, v, w):
        if dist[u] + w < dist[v]: return "negative cycle reachable from s"
    return dist
```

正确性：归纳于边数。第 i 轮结束时，对每个 v，所有"至多 i 条边"的 $s \rightsquigarrow v$ 路径长度都 $\geq$ `dist[v]` 且至少有一条达到。$i = n-1$ 时覆盖所有简单路径（最短路若不含负环，必可化为简单路径）。□

负环检测：第 n 轮仍可松弛 $\iff$ 存在从 s 可达的负环（绕一圈就无限变小）。

复杂度：$O(nm)$。稀疏图（$m = O(n)$）上是 $O(n^2)$，比 Dijkstra 慢；稠密图上 $O(n^3)$ 与 Floyd–Warshall 同阶。

优化：SPFA（队列优化）平均快但最坏仍是 $O(nm)$；实践中若图有负权但无负环，先跑一遍 Bellman–Ford 或 Johnson 重赋权再用 Dijkstra 更快。

8.4 Floyd–Warshall（APSP）

思想：DP。$D^{(k)}[i][j]$ = 只允许经过前 k 个顶点作为中间点时，$i \to j$ 的最短距离。

$$D^{(k)}[i][j] = \min\left(D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\right)$$

```
FloydWarshall(W):                   # W[i][j] = 边权，无边为 INF，W[i][i]=0
    D = W 的副本
    for k = 1..n:
        for i = 1..n:
            for j = 1..n:
                D[i][j] = min(D[i][j], D[i][k] + D[k][j])
    return D
```

正确性：对 k 归纳。$i \to j$ 的最短路要么不经过 k（第一项），要么经过 k，此时可拆成 $i \to k$ 与 $k \to j$ 两段，各自只用前 $k-1$ 个中间点（第二项）。□

复杂度：$O(n^3)$ 时间，$O(n^2)$ 空间。**能处理负边**，但不能有负环（有负环时 $D[i][i] < 0$ 可检测）。

经典反例：把 k 循环写在最内层（`for i, for j, for k`）会算错。因为状态转移要求"所有 i, j 的第 $k-1$ 层已算完"，k 必须是最外层。

8.5 选型速查

条件	选择	复杂度
无权图	BFS	$O(n+m)$
DAG（可负权）	拓扑序 DP	$O(n+m)$
非负权，单源	Dijkstra + 堆	$O((n+m)\log n)$
有负权，单源	Bellman–Ford	$O(nm)$
全对，n 小	Floyd–Warshall	$O(n^3)$
全对，稀疏且非负	每个点跑 Dijkstra	$O(n(n+m)\log n)$
全对，稀疏有负权无负环	Johnson（重赋权 + Dijkstra）	$O(nm + n^2 \log n)$

9. 第 8 章：贪心 I —— 区间调度与 Huffman 编码

9.1 贪心的通用范式与"什么时候能贪"

讲义给的定义很朴素：**贪心算法逐个部件构造解，每一步都选"当下看起来最显然、最直接有利"的那块**。难点从来不是写算法，而是回答两个问题：

1. 选哪个贪心准则？（区间调度有 3 个候选准则，只有 1 个对）
2. 为什么它对？（必须证明，否则只是"看起来对"）

证明贪心的统一框架（讲义明确给出的模式）：

> **归纳假设**：贪心的前 m 个选择，与**某个**最优解的前 m 步一致（等价说法：做完 m 步后，"达到最优"仍然可能）。 **基础**：$m = 0$ 时显然。 **归纳步**：用问题特有的结构，把 OPT 的第 $m+1$ 个选择**替换**成贪心的选择，证明解不变差。

这就是**交换论证（exchange argument）**的骨架。§18 专题会把它抽象成可复用的模板。

9.2 区间调度（Interval Scheduling）

问题：n 个任务，第 i 个占据时间区间 $[s_i, e_i)$。求最大的两两不重叠子集。

三个候选贪心准则，两个错：

准则	反例	结果
最早开始（earliest start）	$[0,10]$ 与 $[1,2],[2,3],\ldots$	选了长的，只能得 1，最优是很多个
最短时长（shortest interval）	$[0,5]$ 与 $[1,2],[3,4]$	选 $[0,5]$ 得 1，最优 2
最早结束（earliest finish）	—	**正确**

```
IntervalScheduling(J):
    按结束时间 e_i 升序排序
    S = []; last_end = -INF
    for j in J (已排序):
        if s_j >= last_end:
            S.add(j); last_end = e_j
    return S
```

正确性（交换论证）：

> **定理**：贪心解 $G = \{g_1, \ldots, g_k\}$（按结束时间排序）是最优的。

证明：设 $O = \{o_1, \ldots, o_m\}$ 是任一最优解，按结束时间排序，且在所有最优解中**与 G 的公共前缀最长**。反设 $G \neq O$，令 i 是第一个 $g_i \neq o_i$ 的下标。

由贪心准则，$e(g_i) \le e(o_i)$（g_i 是所有与 $g_1, \ldots, g_{i-1}$ 兼容的任务里结束最早的，而 o_i 与 $o_1, \ldots, o_{i-1} = g_1, \ldots, g_{i-1}$ 兼容，故是候选）。

构造 $O' = O \setminus \{o_i\} \cup \{g_i\}$。需验证 O' 仍可行： – g_i 与 $o_1, \ldots, o_{i-1}$ 兼容（贪心选择保证）； – g_i 与 $o_{i+1}, \ldots, o_m$ 兼容：因 $e(g_i) \le e(o_i) \le s(o_{i+1})$，且 $s(g_i) \ge e(g_{i-1}) = e(o_{i-1})$。

故 O' 可行且 $|O'| = |O|$，仍最优，但与前缀 $g_1..g_i$ 的公共长度比 O 多 1，与"O 前缀最长"矛盾。□

复杂度：排序 $O(n \log n)$；扫描 $O(n)$。若输入已按结束时间有序，则 $O(n)$。

实现细节：判断兼容用 `s_j >= last_end`，而不是"区间不相交"的一般判定——因为已按结束时间排序，只需和**最后一个被选中**的比。

9.3 Huffman 编码

问题：给 n 个字符及频率 $f_1, \ldots, f_n$，构造**前缀码**（prefix-free code：没有任何码字是另一个的前缀，从而可无歧义解码），使总编码长度 $\sum_i f_i \cdot \ell_i$ 最小。

为什么前缀码：可变长码若无前缀性质，解码需要分隔符或回溯；前缀码让解码变成"沿树走到叶子"，一次扫描完成。

Huffman 算法：

```
Huffman(f_1..f_n):
    Q = 最小堆，每个字符一个叶节点，键为频率
    while |Q| > 1:
        x = Q.extractMin(); y = Q.extractMin()
        建新节点 z，z.freq = x.freq + y.freq
        z.left = x; z.right = y
        Q.insert(z)
    return Q.extractMin()      # 唯一的根，即编码树
```

正确性需要两个引理：

> **引理 1（贪心选择性质）**：存在一棵最优前缀树，其中频率最低的两个字符 a, b 是**兄弟叶**，且深度最大。

证明（交换论证）：设 T 是任一最优树。取深度最大的两个兄弟叶 x, y（深度最大的叶一定成对出现兄弟，否则可把更深的那个上移降低代价）。不妨 $f_a \le f_b \le f_x \le f_y$（$a, b$ 是最低两个频率，x, y 是任意两个最深叶）。交换 a 与 x、b 与 y 的位置，代价变化为

$$\Delta = f_a(d_y - d_a) + f_b(d_x - d_b) \quad \text{（示意）}$$

严格写：交换前贡献 $f_a d_a + f_x d_x$，交换后 $f_a d_x + f_x d_a$，差值 $= (f_a - f_x)(d_x - d_a) \le 0$（因 $f_a \le f_x$ 且 $d_x \ge d_a$）。两次交换都不增代价，故存在最优树使 a, b 在最深处且互为兄弟。□

> **引理 2（最优子结构）**：把 a, b 合并成频率 $f_a + f_b$ 的新字符 c 后，若 T' 是合并后问题的最优树，则把 c 展开成 a, b 得到的 T 是原问题的最优树。

证明：代价关系为 $\mathrm{cost}(T) = \mathrm{cost}(T') + f_a + f_b$（$a, b$ 各深一层）。若 T 不是最优，取最优的 T^*，由引理 1 可设 a, b 为兄弟，收缩得 $T^{*\prime}$，则 $\mathrm{cost}(T^{*\prime}) = \mathrm{cost}(T^*) - (f_a + f_b) < \mathrm{cost}(T) - (f_a + f_b) = \mathrm{cost}(T')$，与 T' 最优矛盾。□

由这两个引理，对字符数归纳即得 Huffman 最优。

复杂度：$n - 1$ 次合并，每次堆操作 $O(\log n)$，共 $O(n \log n)$。若频率已排序，可用两个队列做到 $O(n)$。

经典反例：

反例	说明
固定长度编码	8 个等频字符时定长 3 bit/字符是最优；但频率不均时 Huffman 显著更优（如频率 $0.5, 0.25, 0.125, \ldots$ 时平均长度趋近 1 bit）
按频率从高到低贪心"分配短码"但不管树形	会构造出非前缀码（如 $\{0, 01, 1\}$），解码歧义
Huffman 用于"必须等长"或"有最大码长限制"的场景	Huffman 不保证码长上界；需改用 package-merge 等算法（**推断**：讲义未涉及，属编码理论扩展）
频率为 0 的字符	仍会进树，可先剔除；否则浪费码长

10. 第 9 章：贪心 II —— 最小生成树（MST）

10.1 问题动机

输入：连通无向图 $G = (V, E)$，边权 $w_e \ge 0$。**输出**：连接所有顶点、总权最小的边集。

应用：网络布线的成本最小化（讲义原例：城市间铺路/电力/电话线）、聚类（删去 MST 中最长的 $k - 1$ 条边得 k 簇）、以及作为其它算法的预处理。

先确认一件事：最优解一定是树。若边集连通且有环，删去环上任一边仍连通且代价不增，故最优解无环、连通、$n - 1$ 条边，即生成树。

10.2 两个结构性定理（所有 MST 算法的基石）

定义（割）：把 V 分成两个非空集合 S 与 $V \setminus S$ 称为一个割；跨越割的边集记为 $\delta(S)$。

> **定理 1（割性质，cut property）**：对任意割 $(S, V\setminus S)$，设 e 是 $\delta(S)$ 中权最小的边，则**存在**一棵 MST 包含 e。

证明（交换论证）：设 T 是任一 MST。若 $e\in T$ 则完事。否则 $T+e$ 含唯一环 C，C 必跨越割偶数次，故存在另一条 $e'\in C\cap\delta(S)$，$e'\neq e$。由 e 的最小性 $w_e\le w_{e'}$。令 $T'=T+e-e'$，则 T' 仍是生成树，且 $w(T')=w(T)+w_e-w_{e'}\le w(T)$。由 T 最优知等号成立，T' 也是 MST 且含 e。□

> **定理 2（环性质，cycle property）**：对任意环 C，设 e 是 C 上权**严格最大**的边，则 e 不属于任何 MST。

证明：若 MST T 含 e，删去 e 把 T 分成两个连通块，环 C 上必有另一条边 e' 跨越该分割，$w_{e'}<w_e$，则 $T-e+e'$ 更优，矛盾。□

这两条定理的用法差异很关键：**割性质说"最轻的跨割边可以安全地加"（Kruskal/Prim 的依据），环性质说"最重的环边可以安全地删"（逆向删除算法的依据）。**

10.3 Kruskal：按边权从小到大加，不成环就加

```
Kruskal(G):
    按 w_e 升序排序所有边
    DSU = 并查集，每个顶点独立成集
    T = []
    for (u, v, w) in 排序后的边:
        if find(u) != find(v):        # 不成环
            T.add((u,v)); union(u, v)
            if |T| == n-1: break
    return T
```

正确性：每次加入的边 $e=(u,v)$ 是"连接当前两个不同连通块"的最小边。取 S 为 u 当前所在连通块，则 e 是 $\delta(S)$ 中权最小的边（因为所有更小的边已被处理过；若某更小边跨越 S，它早该把 S 与外界连通，矛盾）。由割性质，存在 MST 含 e。对加入次数归纳即得 T 是 MST。□

复杂度：排序 $O(m\log m)$；m 次并查集操作，带路径压缩 + 按秩合并的并查集单次摊还 $O(\alpha(n))$（α 是反 Ackermann 函数，实际 ≤ 4）。总计

$$O(m\log m+m\,\alpha(n))=O(m\log n)$$

经典反例/陷阱：

- **必须先排序**。若按输入顺序加边，得到的是某个生成树，不是 MST。
- **负权完全没问题**。割性质对任意实数权成立（MST 的贪心是"全局最优"的少数例外之一）。
- **图不连通**：Kruskal 得到**最小生成森林**，需在算法结束时检查 $|T|<n-1$ 并报"不连通"。
- **重边**：无影响，排序后任选其一（权相同，都在某 MST 中）。

10.4 Prim：从一个点开始，每次加最轻的出边

```
Prim(G, r):
    for all v: key[v] = INF, parent[v] = NIL
    key[r] = 0; H = 最小堆 {(0, r)}
    while H 非空:
        d, u = H.extractMin()
        if u in visited: continue
        visited.add(u)
        for (u, v, w) in Adj[u]:
            if v not in visited and w < key[v]:
                key[v] = w; parent[v] = u; H.insert((w, v))
    return {(parent[v], v) : v != r}
```

正确性：设当前已选顶点集为 S（构成一棵树），算法选 $\delta(S)$ 中最轻的边 e。由割性质，存在 MST 含 e。归纳即得。□

复杂度：堆实现 $O((n+m)\log n)$；数组实现 $O(n^2)$，稠密图更优。

Kruskal vs Prim：

	Kruskal	Prim
思路	全局按边权扫描	从根生长
依赖	并查集	优先队列
适合	稀疏图	稠密图（数组版 $O(n^2)$）
不连通图	自然得到森林	只能得到含 r 的分量
并行/分布式	较易（边排序可分块）	较难（依赖全局前沿）

11. 第 10 章：动态规划 I —— 从 Fibonacci 到编辑距离

11.1 问题动机：为什么需要 DP

朴素递归求 Fibonacci 是 $T(n) = T(n-1) + T(n-2)$，即 $T(n) \geq 2^{n/2}$（讲义说 discussion 6 会证）。原因不是"递归慢"，而是**同一个子问题被反复从头算**：递归树里有大量重复节点。

DP 的两个要素（讲义明确列出）：

1. **最优子结构（optimal substructure）**：大问题的解可由子问题的解组合而成。
2. **重叠子问题（overlapping subproblems）**：子问题被反复用到，值得存表。

只有 1 没有 2 是分治（如归并排序，子问题不重叠）；只有 2 没有 1 通常无从下手。**两者都有才是 DP。**

两种实现：

	自顶向下（top-down, memoization）	自底向上（bottom-up, tabulation）
写法	递归 + 缓存	循环填表
优点	只算子问题中真正需要的那些	无递归开销；便于空间优化
缺点	递归深度、哈希开销	可能算用不到的状态
顺序	隐式（调用时）	显式（子问题 DAG 的拓扑序）

子问题 DAG：把每个子问题看成一个节点，若子问题 j 直接用到 i 则连边 $i \to j$。DP 的正确性 = **按该 DAG 的拓扑序计算**；DP 的复杂度 = **状态数 × 单状态转移代价**。

11.2 加权区间调度（DP 的第一课）

问题：与 §9.2 相同的区间，但每个任务有收益 v_i，求**总收益最大**的兼容子集。

贪心失效：最早结束准则不再最优。反例：$[0,3]$ 收益 100，$[0,1]$ 收益 1，$[1,2]$ 收益 1，$[2,3]$ 收益 1。最早结束选 3 个小任务得 3，最优是单个 100。

DP 设计：

1. **排序**：按结束时间排序，$1..n$。
2. **定义 $p(j)$**：最大的 $i < j$ 使 $e_i \leq s_j$（与 j 兼容的最后一个任务）。可用二分查找预处理。
3. **状态**：$OPT(j)$ = 只考虑前 j 个任务时的最大收益。
4. **转移**：

$$OPT(j) = \max\left(OPT(j-1),\ v_j + OPT(p(j))\right), \qquad OPT(0) = 0$$

含义：要么不选 j（退化为前 $j-1$），要么选 j（收益 v_j 加上与 j 兼容的前缀）。

```
WeightedIntervalScheduling(intervals):
    按结束时间排序；对每个 j 二分求 p(j)
    OPT[0] = 0
    for j = 1..n:
        OPT[j] = max(OPT[j-1], v_j + OPT[p(j)])
    # 回溯构造解
    S = []; j = n
    while j > 0:
        if v_j + OPT[p(j)] >= OPT[j-1]: S.add(j); j = p(j)
        else: j = j - 1
    return OPT[n], S
```

正确性（对 j 归纳）：最优解要么不含 j（值 $\le OPT(j-1)$），要么含 j，此时其余任务必须都在 $p(j)$ 之前且兼容，值 $\le v_j + OPT(p(j))$。两种情形都被转移覆盖，且转移的两项都可由可行解达到。□

复杂度：$O(n \log n)$（排序 + n 次二分），DP 本身 $O(n)$。

11.3 最长递增子序列（LIS）

问题：给定数组 $a_1 .. a_n$，求最长的下标递增且值递增的子序列长度。

$O(n^2)$ DP：

$$L(j) = \max\left(1,\ \max_{i<j,\, a_i<a_j} \{L(i)+1\}\right)$$

```
LIS_n2(a):
    L = [1]*n
    for j = 1..n-1:
        for i = 0..j-1:
            if a[i] < a[j]: L[j] = max(L[j], L[i]+1)
    return max(L)
```

$O(n \log n)$ 做法（patience sorting）：维护数组 `tails`，`tails[k]` = 所有长度为 $k+1$ 的递增子序列中**最小可能的结尾值**。对每个 a_j 二分找第一个 $\ge a_j$ 的位置并替换（若都更小则追加）。`tails` 的长度即答案。

> **注意**：`tails` **不是**一个 LIS 本身（这是最常见的误解）。例如 $a = [2,1,3]$，`tails` 最终为 $[1,3]$，恰好是 LIS；但对 $a = [1,3,2]$，`tails` 为 $[1,2]$，而 LIS 是 $[1,3]$ 或 $[1,2]$——长度对，内容可能不对。

经典反例：把 LIS 当成"最长递增**连续**子数组"（那是另一个问题，$O(n)$ 滑动窗口/线性扫描可解）。**子序列（subsequence）不要求连续**。

11.4 编辑距离（Edit Distance）

问题：把字符串 x（长 m）改成 y（长 n），允许三种操作，代价各为 1：插入、删除、替换。求最小总代价。

状态：$E(i,j)$ = 把 $x[1..i]$ 变成 $y[1..j]$ 的最小代价。

转移（按最后一个字符分情况）：

$$E(i,j) = \min \begin{cases} E(i-1,j)+1 & \text{删除 } x_i \\ E(i,j-1)+1 & \text{插入 } y_j \\ E(i-1,j-1) + [x_i \neq y_j] & \text{匹配或替换} \end{cases}$$

边界：$E(0,j) = j$（全插入），$E(i,0) = i$（全删除）。

```
EditDistance(x, y):
    for i = 0..m: E[i][0] = i
    for j = 0..n: E[0][j] = j
    for i = 1..m:
        for j = 1..n:
            E[i][j] = min(E[i-1][j] + 1,
                          E[i][j-1] + 1,
                          E[i-1][j-1] + (x[i] != y[j]))
    return E[m][n]
```

正确性：考虑最优编辑序列对 x_i 与 y_j 的最后一次操作，必属三种之一：删 x_i、插 y_j、或把 x_i 与 y_j 对齐（代价 $[x_i \neq y_j]$）。三种情形分别对应三条转移，且每条转移都能由某个合法编辑序列达到。□

复杂度：$O(mn)$ 时间，$O(mn)$ 空间；滚动数组可降到 $O(\min(m,n))$ 空间（但无法回溯路径）。

应用：diff 工具、拼写纠错、生物序列比对（DNA 的编辑距离即"插入/删除"两操作的 LCS 变体）。

11.5 经典反例：DP 建模的三个常见错误

错误	反例	正确做法
状态定义漏信息	编辑距离若只定义 $E(i)$（不区分 j），无法表达"已匹配到 y 的哪里"	状态必须包含决定未来的**全部**信息
转移漏情形	背包若只考虑"选/不选当前物品"却允许重复选，会漏"选多次"	0/1 背包与完全背包的转移方向相反（见 §12）
计算顺序错	按 i 递增但转移依赖 $E(i,j-1)$ 时，内层必须 j 递增	按子问题 DAG 的拓扑序填表

12. 第 11 章：动态规划 II —— 背包、链矩阵乘、TSP、树上独立集

12.1 0/1 背包（Knapsack）

问题：n 件物品，第 i 件重 w_i、价值 v_i；背包容量 W。每件**至多取一次**，求最大总价值。

为什么贪心失效（必考反例）：按"单位价值最高"贪心。取 $W=10$，物品 $(w,v)=(6,6),(5,5),(5,5)$。单位价值都是 1，贪心先取重量 6 的，剩余 4 装不下任何物品，得 6；最优是取两件重量 5 的，得 10。

DP 设计：

- 状态：$K(i,w)$ = 只考虑前 i 件、容量为 w 时的最大价值。
- 转移（对第 i 件：不取 / 取）：

$$K(i,w)=\max\big(K(i-1,w),\ K(i-1,w-w_i)+v_i\big)\quad (w\geq w_i)$$

- 边界：$K(0,w)=0$，$K(i,w)=-\infty$ 若 $w<0$。

```
Knapsack01(w[1..n], v[1..n], W):
    for w = 0..W: K[0][w] = 0
    for i = 1..n:
        for w = 0..W:
            K[i][w] = K[i-1][w]
            if w >= w[i]: K[i][w] = max(K[i][w], K[i-1][w - w[i]] + v[i])
    return K[n][W]
```

复杂度：$O(nW)$ 时间，$O(W)$ 空间（一维滚动时必须**逆序**枚举 w，否则同一物品会被用多次——这正是"完全背包"的写法）。

这是伪多项式（pseudo-polynomial）算法：$O(nW)$ 看起来是多项式，但 W 是**数值**不是输入长度；W 的二进制编码长度是 $\log W$，所以对输入规模而言是 $O(n2^{\log W})$。这就是背包是 NP 难（第 15 章）却不与"$O(nW)$ 算法存在"矛盾的原因。

变体	转移方向	说明
0/1 背包	w **逆序**	每件至多一次
完全背包	w **正序**	每件无限次
多重背包	加一层件数循环 / 二进制拆分	每件有上限 c_i

12.2 链矩阵乘法（Chain Matrix Multiplication）

问题：连乘 $A_1A_2\cdots A_n$，A_i 的维度是 $d_{i-1}\times d_i$。矩阵乘法满足结合律，不同的加括号顺序代价差别巨大。求最小标量乘法次数。

例：A_1 是 10×100，A_2 是 100×5，A_3 是 5×50。 - $(A_1A_2)A_3$：$10\cdot 100\cdot 5+10\cdot 5\cdot 50=5000+2500=7500$。 - $A_1(A_2A_3)$：$100\cdot 5\cdot 50+10\cdot 100\cdot 50=25000+50000=75000$。

差 10 倍。这个数字在深度学习编译器（算子融合、attention 的 QK^T 与 $(QK^T)V$ 顺序）里是实打实的性能差异。

DP 设计：

- 状态：$C(i,j)$ = 乘 $A_i \cdots A_j$ 的最小代价（$i \le j$）。
- 转移（枚举最后一次分割点 k）：

$$C(i,j) = \min_{i \le k < j} \{ C(i,k) + C(k+1,j) + d_{i-1} d_k d_j \}$$

- 边界：$C(i,i) = 0$。
- 计算顺序：**按区间长度递增**（因为 $C(i,j)$ 依赖更短的区间）。

```
ChainMatrix(d[0..n]):
    for i = 1..n: C[i][i] = 0
    for len = 2..n:
        for i = 1..n-len+1:
            j = i + len - 1
            C[i][j] = INF
            for k = i..j-1:
                C[i][j] = min(C[i][j], C[i][k] + C[k+1][j] + d[i-1]*d[k]*d[j])
    return C[1][n]
```

复杂度：$O(n^3)$ 时间（$O(n^2)$ 个状态，每个 $O(n)$ 转移），$O(n^2)$ 空间。

经典反例：若按 i 从 1 到 n、j 从 i 到 n 的双重循环填表，会用到尚未计算的 $C(k+1,j)$（当 $k+1 > i$ 时区间更短但 i 更大）。**必须按长度枚举**，这是"计算顺序必须匹配子问题 DAG 拓扑序"的最好例子。

12.3 旅行商问题（TSP）的 Held–Karp DP

问题：n 个城市，两两距离 d_{ij}，从城市 1 出发经过所有城市恰好一次并回到 1，求最短闭合回路。

暴力枚举 $(n-1)!$ 条回路；$n = 25$ 时约 10^{25}。Held–Karp 把它压到 $O(n^2 2^n)$，$n = 25$ 时约 10^{10}——仍然大，但已是"能算"的范围。

状态：$T(S, j)$ = 从 1 出发、恰好访问集合 S（含 1 与 j）中所有城市、最后停在 j 的最短路径长度。

转移：

$$T(S,j) = \min_{i \in S,\, i \ne j} \{ T(S \setminus \{j\}, i) + d_{ij} \}$$

边界：$T(\{1\}, 1) = 0$；$T(S, 1) = \infty$ 对 $|S| \ge 2$（不允许中途回到 1）。

答案：$\min_{j \ne 1} \{ T(\{1..n\}, j) + d_{j1} \}$。

```
TSP(d):
    for all S, j: T[S][j] = INF
    T[{1}][1] = 0
    for s = 2..n:                       # 按集合大小递增
        for S with |S| = s and 1 in S:
            for j in S, j != 1:
                for i in S, i != j:
                    T[S][j] = min(T[S][j], T[S\{j}][i] + d[i][j])
    return min over j != 1 of (T[full][j] + d[j][1])
```

复杂度：状态数 $2^{n-1} \cdot n$，每个状态 $O(n)$ 转移 → $O(n^2 2^n)$ 时间，$O(n 2^n)$ 空间。

正确性：对 $|S|$ 归纳。最优路径的最后一条边必是某个 $i \to j$，去掉 j 后是 $S \setminus \{j\}$ 上的最优路径（否则替换成更优的会改进整体）。□

这说明了什么：TSP 有 $O(n^2 2^n)$ 的精确算法，所以它"只"是 NP 难而非不可解；2^n 因子是本质的（**推断**：在 ETH（Exponential Time Hypothesis）下 TSP 不存在 $2^{o(n)}$ 算法，这是超越本课的研究结论）。

12.4 树上的最大独立集

问题：无向图 $G = (V, E)$，求最大独立集（两两不相邻的顶点集）。一般图上是 NP 难的（第 15 章），但**树上有多项式 DP**。

设计：任取根 r，对每个节点 u 定义两个状态：

- $I(u, 0)$：u **不选**时，子树 T_u 内的最大独立集大小；
- $I(u, 1)$：u **选**时，子树 T_u 内的最大独立集大小。

转移（u 选则所有孩子都不能选；u 不选则孩子可选可不选）：

$$I(u,1) = 1 + \sum_{c \in \text{children}(u)} I(c,0), \qquad I(u,0) = \sum_{c} \max\big(I(c,0), I(c,1)\big)$$

答案：$\max(I(r,0), I(r,1))$。**计算顺序**：DFS 后序（自底向上）。

```
TreeMIS(root):
    postorder = DFS 后序
    for u in postorder:
        I[u][1] = 1; I[u][0] = 0
        for c in children(u):
            I[u][1] += I[c][0]
            I[u][0] += max(I[c][0], I[c][1])
    return max(I[root][0], I[root][1])
```

复杂度：$O(n)$ 时间（每个节点常数工作），$O(n)$ 空间。

经典反例：把同一 DP 直接用到一般图上会失败——因为一般图有环，"子树"不存在，状态会循环依赖。树结构是这里可解性的**唯一**来源。（有环图上的 MIS 是 NP 难的；二分图上的 MIS 可多项式求解，因为 MIS = n − 最小顶点覆盖，而二分图的最小顶点覆盖 = 最大匹配，见 §14.5。）

13. 第 12 章：线性规划（Linear Programming）

13.1 问题动机

很多优化问题的目标与约束都是**线性**的：排产、配比、调度、投资组合、以及本课后面要用的最大流、最短路、二分匹配、零和博弈。线性规划（LP）是它们的统一语言，而且**有 polynomial 时间的算法**（椭球法 1979、内点法 1984），所以"能写成 LP"就等于"能高效求解"。

讲义的引入例子是"John 的面包店"：甜甜圈每个赚 \$5、蛋糕每个赚 \$25；面粉、糖、鸡蛋的库存有限，每种产品消耗量给定；问各做多少使利润最大。设 x 为甜甜圈数量、y 为蛋糕数量：

$$\max\ 5x + 25y \quad \text{s.t.} \quad 2x + 5y \le 200,\ 2x + 9y \le 300,\ 7x + 12y \le 500,\ x, y \ge 0$$

注意 x, y 必须是整数才有物理意义（不能做 1.37 个蛋糕）——**这个"整数约束"正是 LP 与 ILP 的分水岭**，也是 NP 完全性的来源之一（第 15 章）。LP 的可行域是凸多面体，最优解必在顶点，而顶点坐标可能非整数。

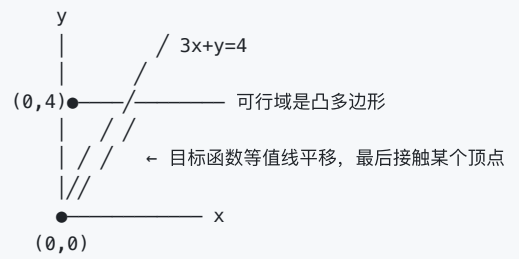
13.2 标准形式与几何

标准形式（不同教材写法不同，本质等价）：

$$\min\ c^T x \quad \text{s.t.} \quad Ax \le b,\ x \ge 0$$

几何事实（讲义用二维图直观展示）：

1. 每个线性约束定义一个**半空间**（halfspace）；
2. 可行域 = 半空间之交 = **凸多面体**（polytope）；
3. 目标函数的**等值线**是超平面，沿梯度方向平移直到"离开"可行域；
4. 最优解（若存在）必在**顶点**（vertex）达到；若两个顶点同时最优，则整条边都最优。



三种可能的结果：有唯一最优解、有无穷多最优解（整条边）、**无界**（unbounded）或**不可行**（infeasible）。LP 求解器必须能区分这四种情形（含"有多解"）。

13.3 单纯形法（Simplex）

思想：从一个顶点出发，沿"使目标改善"的边走到相邻顶点，直到没有改善方向。

```
Simplex(A, b, c):
    把 LP 转成标准形，引入松弛变量，得初始基本可行解（BFS）
    while 存在使目标改善的相邻顶点:
        选一个进基变量（pricing，如 Bland 规则或最大改进）
        选一个出基变量（ratio test，保证可行性不破坏）
        做枢轴变换（pivot），移动到相邻顶点
    return 当前顶点
```

复杂度：最坏**指数**（Klee–Minty 立方体构造，n 维超立方体上单纯形走 2^n 个顶点）；但实践中迭代次数通常与 n, m 近似线性，是 20 世纪最成功的"理论坏、实践好"算法之一。1979 年 Khachiyan 的椭球法首次给出多项式时间（$O(n^6 L^2)$ 量级，实际极慢）；1984 年 Karmarkar 的内点法既有多项式保证又快，现代求解器多为**内点法 + 单纯形交叉验证**。

经典反例：

反例	说明
Klee–Minty 立方体	单纯形最坏 2^n 步；说明"平均快"不等于"最坏快"
退化（degenerate）顶点	可能死循环（循环枢轴）；Bland 规则可保证终止但更慢
无界 LP	目标沿某方向无限改善，求解器须检测并报 unbounded
把 ILP 当 LP 解	得到 $x = 1.5$ 这类分数解，不能直接取整（见 §13.5）

13.4 用 LP 表达经典问题

问题	LP 形式	备注
最大流	$\max \sum_v f_{sv}$ s.t. 容量约束 + 流量守恒 + $f \geq 0$	见 §14；约束矩阵是全幺模的，故有整数最优解
最短路	$\max\ d_t$ s.t. $d_v \leq d_u + w_{uv}$，$d_s = 0$	差分约束系统；对偶是"最小割/势能"形式
二分匹配	$\max \sum x_{uv}$ s.t. 每个点度数 ≤ 1，$0 \leq x \leq 1$	全幺模 → 有 0/1 最优解
最小顶点覆盖	$\min \sum x_v$ s.t. $x_u + x_v \geq 1\ \forall (u,v)$，$0 \leq x_v \leq 1$	一般图上 LP 有分数解（三角形：$x = 1/2$ 全取，值 1.5 < 2）
最大独立集	$\max \sum x_v$ s.t. $x_u + x_v \leq 1\ \forall (u,v)$	同上，LP 松弛不紧
零和博弈	$\max v$ s.t. $\sum_i x_i A_{ij} \geq v$，$\sum x_i = 1$，$x \geq 0$	见 §15；对偶即"对手的最小最大"

全幺模（total unimodularity）：若约束矩阵 A 的每个方子阵行列式为 $0, \pm 1$，则 LP 的顶点自动是整数点，LP 松弛**无损**。网络流矩阵、二分图匹配矩阵都是全幺模的——这就是"最大流有整数解""二分匹配可用 LP 解"的深层原因。

13.5 整数规划（ILP）与松弛间隙

把 $x \in \{0, 1\}$ 换回 $0 \leq x \leq 1$ 得到 LP **松弛**。松弛的最优值是原问题的**上界**（最大化问题），两者之差叫**整数间隙（integrality gap）**。

反例（三角形上的最大独立集）：K_3 的 ILP 最优是 1（只能选一个点）；LP 松弛取 $x_1 = x_2 = x_3 = 1/2$，目标 1.5，所有约束 $x_u + x_v \leq 1$ 满足。间隙 $1.5/1 = 1.5$。

反例（三角形上的最小顶点覆盖）：ILP 最优 2；LP 松弛全取 $1/2$ 得 1.5。间隙 $2/1.5 = 4/3$。

这两个反例说明：**"把 ILP 松弛成 LP 再取整"不是万能解**，取整后可能不可行，也可能差很多。这正是近似算法要单独研究的原因（§20）。

13.6 LP 的建模陷阱

陷阱	例	修法
遗漏非负约束	变量默认可为负，得到不合物理意义的解	显式写 $x \geq 0$
用线性式表达"或"	"要么选 A 要么选 B" 不是线性的	需要 0/1 变量 + 大 M 法（引入 ILP）
目标里的绝对值	$\min\sum	x_i - t

陷阱	例	修法
约束写反	$\geq$ 与 $\leq$ 混淆导致无界	用小实例手算验证可行域
忽略整数性	直接对 LP 解四舍五入	检查可行性；用 ILP 或近似算法

14. 第 13 章：最大流与最小割

14.1 问题动机与定义

输入：有向图 $G=(V,E)$，源 s、汇 t，每条边容量 $c_e \in \mathbb{Z}_{\geq 0}$（或实数）。 **输出**：从 s 到 t 的最大流量。

讲义的历史引子很具体：Harris 与 Ross 在 1950 年代研究苏联铁路网的最大运量（讲义给的总容量是 163,000 吨），他们用了一个叫 "flooding" 的贪心方法，有时给出**错误**答案；于是找 Ford 与 Fulkerson，后者发明了 Ford–Fulkerson 算法。

流（flow）的定义：给每条边赋值 f_e，满足

1. **容量约束**：$0 \leq f_e \leq c_e$；
2. **流量守恒**：对每个 $v \neq s,t$，$\sum_{u \to v} f_{uv} = \sum_{v \to w} f_{vw}$（流入 = 流出）。

流的大小：$|f| = \sum_v f_{sv} = \sum_v f_{vt}$（后者由守恒推出）。

14.2 为什么"贪心找路径"会错

```
    s ──1──► A ──1──► t
    │        │
    1        1
    ▼        ▼
    B ──1──► t          (A→B 也有一条容量 1 的边)
```

若贪心先选路径 $s \to A \to B \to t$，三条边全部饱和，此时 $s \to A$ 与 $B \to t$ 都满了，找不到新路径，得到流量 1。但最优是 2：$s \to A \to t$ 与 $s \to B \to t$。

关键洞察：要得到最优解，必须允许"**撤销**"已做出的决定——把 $A \to B$ 上那 1 单位流量退回去。这就是**残量图（residual graph）**的作用：为每条边 (u,v) 加一条反向边 (v,u)，容量等于当前流量 f_{uv}，表示"最多能退回多少"。

14.3 Ford–Fulkerson 算法

```
FordFulkerson(G, s, t):
    for all e: f_e = 0
    构造残量图 G_f：对每条原边 (u,v)，正向容量 c_e − f_e，反向容量 f_e
    while G_f 中存在 s→t 路径 P:            # 增广路径
        bottleneck = min{ 残量容量 along P }
        沿 P 增广 bottleneck 单位：
            正向边 f += bottleneck；反向边 f −= bottleneck
        更新 G_f
    return f
```

要点：

- 残量图里"反向边"不是原图的边，它只是记账用的。
- 每次增广后，$|f|$ 严格增加（bottleneck >0，容量为整数时至少增 1），所以整数容量下必然终止。
- 终止时**不存在增广路径**，此时流即最大（下面证明）。

14.4 最大流最小割定理

定义（割）：把 V 分成 S（含 s）与 $T = V \setminus S$（含 t）。割的**容量** $c(S,T) = \sum_{u \in S, v \in T} c_{uv}$（只算 $S \to T$ 方向）。

> **定理（Max-Flow Min-Cut）**：$\max_f |f| = \min_{(S,T)} c(S,T)$。

证明分三步。

第一步（弱对偶）：任意流 ≤ 任意割。

对任意流 f 与割 (S,T)：

$$|f| = \sum_{v\in T} f_{sv} - \sum_{v\in T} f_{vs} \ \text{（净流出）} = \sum_{u\in S, v\in T} f_{uv} - \sum_{u\in S, v\in T} f_{vu}$$

（第二个等号由对 S 内所有点求守恒式并相加得到。）因为 $f_{uv} \le c_{uv}$ 且 $f_{vu} \ge 0$，

$$|f| \le \sum_{u\in S, v\in T} c_{uv} = c(S,T)$$

所以**最大流 ≤ 最小割**。□

第二步（构造性）：若 f 无增广路径，则存在割使 $|f| = c(S,T)$。

在残量图 G_f 中，令 S = 从 s 可达的顶点集。由假设 $t \notin S$，故 (S,T) 是合法割。对任意 $u \in S, v \in T$：

- 若原边 (u,v) 存在，则残量容量 $c_{uv} - f_{uv} = 0$，即 $f_{uv} = c_{uv}$（否则 v 可达，矛盾）；
- 若原边 (v,u) 存在，则残量容量 $f_{vu} = 0$。

于是

$$|f| = \sum_{u\in S, v\in T} f_{uv} - \sum_{u\in S, v\in T} f_{vu} = \sum_{u\in S, v\in T} c_{uv} - 0 = c(S,T)$$

□

第三步（合并）：设 f^* 是 FF 终止时的流，(S,T) 由第二步给出。则 $|f^*| = c(S,T) \ge \min$ 割 $\ge \max$ 流 $\ge |f^*|$，所有不等号取等，故 f^* 最大且 (S,T) 最小。□

这个证明的结构值得记住：先用"弱对偶"给出上界，再用"算法终止状态"构造一个达到上界的解。**LP 对偶的证明是同一个套路**（§15）。

14.5 二分匹配归约到最大流

问题：二分图 $G = (L, R, E)$，$|L| = |R| = n$，求最大匹配（两两不共享端点的边集）。

构造：加源 s、汇 t；$s \to$ 每个 $l \in L$ 容量 1；每个 $r \in R \to t$ 容量 1；原边 $l \to r$ 容量 1（或 ∞）。

> **定理**：G 有大小为 k 的匹配 $\iff$ 构造的网络有值为 k 的流。

证明：（⇒）匹配的每条边放 1 单位流，加 $s \to l$、$r \to t$，容量与守恒都满足。（⇐）整数容量下 FF 给出**整数流**（每次增广 bottleneck 是整数），每条边流量 $\in \{0, 1\}$；由容量 1 与守恒，每个 l 至多一条出边有流、每个 r 至多一条入边有流，这些边构成匹配。□

整数性引理：若所有容量是整数，Ford–Fulkerson 得到的流是整数流。*证明*：初始流为 0（整数），每次增广的 bottleneck 是整数容量的差，仍为整数，归纳即得。□

两个经典定理（可作为练习）：

- **Hall 定理**：二分图有完美匹配 $\iff$ 对任意 $S \subseteq L$，$|N(S)| \ge |S|$。
- **König 定理**：二分图中最大匹配大小 = 最小顶点覆盖大小。（一般图上不成立；一般图的最大匹配有多项式算法——Blossom 算法，但不在本课范围。）

14.6 复杂度与经典反例

实现	复杂度	说明
Ford–Fulkerson（DFS 找路径）	$O(m \cdot C)$，C 为最大流值	容量大时很慢；C 不是输入长度
Edmonds–Karp（BFS 找最短路）	$O(nm^2)$	与容量无关
Dinic	$O(n^2 m)$	分层图 + 阻塞流，实践中很快
Push–relabel	$O(n^3)$	不同思路（预流推进）

经典反例 1（FF 的指数级慢）：经典构造中，若每次增广都选到瓶颈为 1 的"绕远路"路径，需要 $2C$ 次增广。容量用二进制写只有 $O(\log C)$ 位，所以是**伪多项式**。改用 BFS（Edmonds–Karp）可消除对 C 的依赖。

经典反例 2（无理容量不终止）：若容量是无理数（如含 $\sqrt{2}$），Ford–Fulkerson 可能**永不终止**，且收敛到非最大流（Zwick 的构造）。这是"整数性假设不可省"的经典例子。CS170 通常在整数容量下讨论，但要知道这个边界。

经典反例 3（无向图 vs 有向图）：无向边要建成**两条**反向的有向边，各带该容量；若只建一条，会漏掉反方向流量。

15. 第 14 章：对偶与零和博弈

15.1 问题动机

上一章我们证了"最大流 = 最小割"。这不是流算法独有的巧合，而是**线性规划对偶（duality）**的一个特例。DPV 把对偶称为"一个魔法"（讲义原话：The book calls duality a magic trick）。本章把这个"魔法"讲清楚：**每个最大化问题都有一个"影子"最小化问题，两者的最优值相等。**

对偶的价值有三层：

1. **证明最优性**：给出一个对偶可行解，就等于给出了原问题的上界证书（最大流用最小割证明最优，就是这个道理）。
2. **算法设计**：对偶问题可能更容易求解，或给出更好的近似/定价解释。
3. **理论统一**：最大流最小割、最短路、二分匹配、零和博弈全是同一个定理的化身。

15.2 标准形与对偶的构造规则

原问题（primal，P）：

$$\max\ c^T x \quad \text{s.t.} \quad Ax \le b,\ x \ge 0$$

对偶问题（dual，D）：

$$\min\ b^T y \quad \text{s.t.} \quad A^T y \ge c,\ y \ge 0$$

构造规则（对称形式，最常用）：

原问题（max）	对偶（min）
第 i 个约束 $\le b_i$	第 i 个变量 $y_i \ge 0$
第 j 个变量 $x_j \ge 0$	第 j 个约束 $\ge c_j$
目标系数 c_j	约束右端 c_j
约束右端 b_i	目标系数 b_i
约束矩阵 A	A^T

记忆法："max 配 $\le$ 配非负，转置后 min 配 $\ge$ 配非负"；等式约束对应自由变量（无符号限制），反之亦然。

15.3 弱对偶与强对偶

> **定理 1（弱对偶）**：若 x 是 P 的可行解、y 是 D 的可行解，则 $c^T x \le b^T y$。

证明：$c^T x \le (A^T y)^T x = y^T A x \le y^T b = b^T y$。第一个不等号用 $A^T y \ge c$ 与 $x \ge 0$；第二个用 $Ax \le b$ 与 $y \ge 0$。□

推论：弱对偶立刻给出"任何对偶可行解都是原问题上界的证书"。**最大流最小割就是弱对偶**：任意流 $\le$ 任意割。

> **定理 2（强对偶）**：若 P 与 D 都有可行解，则两者的最优值相等。

证明超出本课范围（要用 Farkas 引理或单纯形终止性），但**必须记住结论与前提**：强对偶要求**两边都可行**。若 P 无界，则 D 不可行；若 P 不可行，则 D 或不可行或无界。

15.4 互补松弛（complementary slackness）

设 x^*, y^* 分别是 P、D 的最优解，则

$$x_j^* > 0 \implies (A^T y^*)_j = c_j, \qquad y_i^* > 0 \implies (Ax^*)_i = b_i$$

直觉：若某资源有剩余（约束不紧），它的影子价格必为 0；若某活动真的在做（变量为正），它的"成本"必恰好等于"收益"。

用途：验证最优性（给出 x, y 后检查互补松弛即可，不必重解）；从对偶解读出经济解释。

15.5 例子：面包店的对偶

原问题（§13.1）：$\max 5x + 25y$ s.t. 面粉 $2x + 5y \le 200$、糖 $2x + 9y \le 300$、蛋 $7x + 12y \le 500$，$x, y \ge 0$。

对偶：设 $p_f, p_s, p_e \ge 0$ 为三种原料的"影子价格"，

$$\min\ 200p_f + 300p_s + 500p_e \quad \text{s.t.} \quad 2p_f + 2p_s + 7p_e \ge 5, \quad 5p_f + 9p_s + 12p_e \ge 25$$

解释：一个"收购商"想买断全部原料，出价 p_f, p_s, p_e；他必须让"生产一个甜甜圈的原料价值 $\ge$ 甜甜圈的利润 \$5"（否则面包店宁愿自己做），同理对蛋糕。他要最小化总支出。**强对偶说：面包店的最大利润 = 收购商的最小支出。** 这就是"影子价格"的经济含义。

（讲义另有"教室分配"的例子：课程到教室的分配，最大化分配数量，本质是二分匹配的 LP，对偶给出"最少需要多少个约束就能证明最优"的解释。）

15.6 零和博弈与 minimax

设定：两人零和博弈，行玩家选策略 $i \in \{1..m\}$，列玩家选 $j \in \{1..n\}$，收益矩阵 A，行玩家得 A_{ij}，列玩家得 $-A_{ij}$。允许**混合策略**（概率分布 $x \in \Delta_m$、$y \in \Delta_n$）。

行玩家的 LP（最大化最坏情形收益 v）：

$$\max\ v \quad \text{s.t.} \quad \sum_i x_i A_{ij} \ge v\ \ \forall j, \quad \sum_i x_i = 1, \quad x \ge 0$$

列玩家的 LP（最小化行玩家的最好情形）：

$$\min\ u \quad \text{s.t.} \quad \sum_j A_{ij} y_j \le u\ \ \forall i, \quad \sum_j y_j = 1, \quad y \ge 0$$

> **定理（von Neumann minimax）**：$\max_x \min_y\ x^T A y = \min_y \max_x\ x^T A y$。

证明思路：这两个 LP 互为对偶（把 v 移项、把等式约束拆成两个不等式即可验证），由强对偶得最优值相等，即 minimax 定理。□

意义："**随机化能消除先手劣势**"。纯策略下"石头剪刀布"没有均衡值，混合策略下均衡值是 0。这也是"随机算法"这一整块内容的思想源头之一：随机性把"可被针对的确定性"变成"不可被针对的分布"。

15.7 经典反例

反例	说明
对偶写错符号	把 $\ge$ 写成 $\le$ 会得到"弱对偶方向相反"，出现 $c^T x \le b^T y$ 被违反的假象
强对偶无条件成立	错。P 无界时 D 不可行；必须检查两边可行性
原问题有整数约束时强对偶仍成立	错。ILP 与其 LP 松弛之间有间隙（§13.5），**对偶只对 LP 成立**
把"影子价格"当真实价格	影子价格是边际价值，只在当前最优基附近有效（灵敏度范围外会变）
最大流最小割直接推广到多商品流	错。多商品流（multi-commodity flow）没有整数最大流最小割定理，且 LP 不等于组合最优

16. 第 15 章：P、NP 与 NP 完全性

16.1 问题动机：为什么需要"难"的理论

到此为止，每个问题我们都能给出多项式算法。但现实中有一大批重要问题（排课、装箱、路径规划、电路布线、蛋白质折叠）几十年没人找到多项式算法。**要么我们不够聪明，要么它们本质上没有多项式算法**。NP 完全性理论把这句话变成数学命题：

> 若**任意一个** NP 完全问题有多项式算法，则**所有** NP 问题都有多项式算法（即 P = NP）。

于是"这个问题是 NP 完全的"成为一个**可操作的停止信号**：不要再找精确多项式算法了，转去做近似、随机化、或用指数算法处理小实例。

16.2 定义

判定问题（decision problem）：输出为 YES/NO 的问题。优化问题可以先转成判定问题（"是否存在长度 $\leq k$ 的回路"）再讨论难度。

P：存在多项式时间算法求解的判定问题类。讲义的说法是"efficiently solvable = 能在多项式时间内求解"，并注明"理论上如此；实践中有时候 $O(n)$ 才算高效"。

NP：存在多项式时间**验证器**（verifier）的判定问题类。即：对 YES 实例，存在一个多项式长度的**证书**（certificate / witness），能在多项式时间内验证它确实使答案为 YES。

问题	证书	验证
3-着色	一个具体着色 $C : V \to \{R, G, B\}$	检查每条边两端异色，$O(m)$
哈密顿回路	一个顶点排列	检查是否成环且覆盖所有点，$O(n)$
SAT	一个真值赋值	代入检查每个子句，$O(\text{公式长度})$
子集和	一个子集	求和比对，$O(n)$
整数分解	两个因子 p, q	相乘比对，$O(n^2)$（注意：分解**不被认为**是 NP 完全的）

关键区分：NP 不是"非多项式"（non-polynomial），而是"**非确定性**多项式"（nondeterministic polynomial）。P $\subseteq$ NP 显然（能解就能验证：忽略证书直接算）。**P = NP 是开放问题**，也是千禧年大奖问题之一。

coNP：NO 实例有短证书的问题类（如"公式不可满足"）。若 P = NP 则 P = NP $\cap$ coNP，但反之未知。

16.3 归约：方向决定一切

> **定义（多项式时间归约）**：$A \leq_p B$ 指存在多项式时间可计算的函数 f，使对任意输入 x：$x \in A \iff f(x) \in B$。

读法："A 归约到 B"意味着"B **至少和** A **一样难**"（B 能解则 A 能解）。所以 $A \leq_p B$ 是"A 不比 B 难"。

这是全课程最容易搞反的地方。记法：

```
A ≤p B     读作 "A reduces to B"
含义：用 B 的算法可以解 A
推论：若 B ∈ P，则 A ∈ P；若 A 难，则 B 难
用途：证明 B 难时，从已知难的 A 归约到 B
```

> **定义（NP 难 / NP 完全）**：– B 是 **NP 难（NP-hard）**：对所有 $A \in$ NP，$A \leq_p B$。– B 是 **NP 完全（NP-complete）**：$B \in$ NP **且** B 是 NP 难。

证明某问题 B 是 NP 完全的模板（两步，缺一不可）：

1. $B \in$ NP：给出证书格式 + 多项式验证算法。
2. B **是 NP 难**：选一个已知 NP 完全问题 A，构造 $A \leq_p B$，并证明**两个方向**：
3. 完备性（completeness）：A 的 YES 实例 $\Rightarrow$ B 的 YES 实例；
4. 可靠性（soundness）：B 的 YES 实例 $\Rightarrow$ A 的 YES 实例。
5. **归约是多项式的**：f 的计算时间 + $f(x)$ 的规模都是多项式。

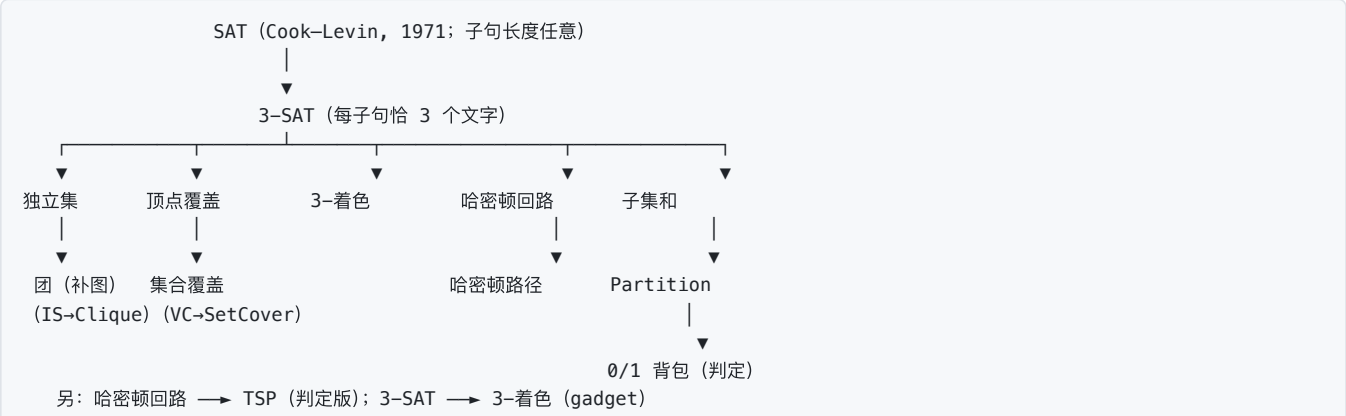
第 2 步的两个方向**都要证**，只证一半是最常见的失分点。

16.4 Cook–Levin 定理与归约图谱

> **定理（Cook 1971, Levin 1973）**：SAT（布尔公式可满足性）是 NP 完全的。

证明思路：任取 NP 问题 A 与它的多项式验证器 V，把"V 接受 (x, w)"编码成一个布尔公式（变量表示图灵机的纸带单元与状态，子句表示转移的局部一致性）。公式长度是 $|x|$ 的多项式，且可满足 $\iff$ 存在证书 w。□

有了 SAT 这个"锚点"，后续所有 NP 完全性都靠**链式归约**：



记忆要点：箭头方向 = 归约方向 = "从难到易证明"的方向。**SAT 是根，3-SAT 是所有实际归约的起点**（因为 3-SAT 的子句结构规整，好构造 gadget）。

16.5 四个必会归约的完整构造

16.5.1 SAT $\leq_p$ 3-SAT

把每个子句"拆短"：

子句长度	转换
$k = 3$	不变
$k > 3$：$(a_1 \vee \cdots \vee a_k)$	引入新变量 $y_1 .. y_{k-3}$，写成 $(a_1 \vee a_2 \vee y_1)(\neg y_1 \vee a_3 \vee y_2) \cdots (\neg y_{k-3} \vee a_{k-1} \vee a_k)$
$k = 2$：$(a \vee b)$	$(a \vee b \vee y)(a \vee b \vee \neg y)$
$k = 1$：(a)	$(a \vee y \vee z)(a \vee \neg y \vee z)(a \vee y \vee \neg z)(a \vee \neg y \vee \neg z)$

正确性直觉：长子句的 y_i 链强制"从某个位置开始全真、之前全假"，从而等价于"至少一个 a_i 为真"。新增变量数是 $O(k)$，公式长度多项式增长。□

16.5.2 3-SAT $\leq_p$ 独立集（Independent Set）

构造：设公式有 m 个子句。

1. 每个子句 $C_j = (a \vee b \vee c)$ 建一个**三角形**（3 个顶点两两相连），三个顶点标记为该子句的三个文字。
2. 对每一对**互补文字**（x 与 $\neg x$，出现在不同子句的顶点），连一条边。

定理：公式可满足 $\iff$ 图有大小为 m 的独立集。

证明：（$\Rightarrow$）给定满足赋值，每个子句至少一个文字为真；从每个三角形里选一个为真的文字顶点，共 m 个。同一三角形内只选一个（无边）；不同三角形之间，被选的两个文字不可能互补（因为它们同时为真），故无边。是独立集。（$\Leftarrow$）给定大小 m 的独立集。每个三角形内至多选一个（三角形是完全图），共 m 个三角形，故**恰好每三角形选一个**。令被选文字为真、其否定为假——由于独立集内无互补文字对，这个赋值**不冲突**。每个子句至少有一个为真文字，故公式可满足。□

规模：$3m$ 个顶点，$3m + \binom{?}{?}$ 条边，多项式。□

16.5.3 独立集 $\leq_p$ 顶点覆盖（Vertex Cover）

事实（互补关系）：对任意图 $G = (V, E)$，S 是独立集 $\iff$ $V \setminus S$ 是顶点覆盖。

证明：S 是独立集 $\iff$ 每条边至少有一个端点在 $V \setminus S$ $\iff$ $V \setminus S$ 覆盖所有边。□

推论：G 有大小 $\geq k$ 的独立集 $\iff$ G 有大小 $\leq n - k$ 的顶点覆盖。

注意：这是**同一个图上的恒等变换**，不需要构造新图——所以归约是 $O(1)$ 的（只改参数）。这也解释了为什么"最大独立集"与"最小顶点覆盖"的难度完全等价（在一般图上都是 NP 难的）。

16.5.4 独立集 $\leq_p$ 团（Clique）

构造：取补图 $\bar{G}$（边 $(u,v) \in \bar{G} \iff (u,v) \notin G$）。

定理：G 有大小 k 的独立集 $\iff$ $\bar{G}$ 有大小 k 的团。

证明：S 在 G 中两两无边 $\iff$ S 在 $\bar{G}$ 中两两有边 $\iff$ S 是 $\bar{G}$ 的团。□

16.5.5 3-SAT $\leq_p$ 子集和（Subset Sum）

构造：设公式有 n 个变量、m 个子句。用 $n+m$ 位十进制数（每位的和不超过 6，不会进位）：

- 对每个变量 x_i，造两个数 $v_i, \bar{v}_i$：第 i 位（变量位）为 1；对每个包含 x_i 的子句 j，第 $n+j$ 位（子句位）为 1（$\bar{v}_i$ 对应 $\neg x_i$）。
- 对每个子句 j，造**两个**哨兵数 s_j, s'_j：只在第 $n+j$ 位为 1。
- **目标 t**：变量位全 1，子句位全 3。

定理：公式可满足 $\iff$ 存在子集和为 t。

证明：（$\Rightarrow$）赋值选真则取 v_i，否则取 $\bar{v}_i$；变量位恰为 1。每个子句至少一个真文字，故该子句位已贡献 1 或 2 或 3；用哨兵补到恰好 3（每子句有两个哨兵可用）。（$\Leftarrow$）和为 t 意味着每个变量位恰好 1（故恰选了 v_i 或 $\bar{v}_i$ 之一，定义赋值），且每个子句位恰为 3，说明至少一个真文字被选中（哨兵最多贡献 2）。□

为什么用十进制而不是二进制：十进制位之间不产生进位，编码与验证都直观；位数 $n+m$ 是多项式，数值大小 10^{n+m} 是**指数级数值**——这正说明子集和是"弱 NP 难"（有伪多项式算法，见 §12.1 背包）。

16.6 其它经典结果（知道结论与直觉即可）

问题	结论	归约直觉
3-着色（3-Coloring）	NP 完全	用基三角形固定 T/F/B 三色，变量用"颜色对"，子句用 OR-gadget 强制至少一端为真
哈密顿回路（Hamiltonian Cycle）	NP 完全	变量 gadget 是"可选择走法"的链，子句 gadget 是必须被某条链"访问"的节点
哈密顿路径	NP 完全	由哈密顿回路加一个"强制起点"的 gadget 得到
集合覆盖（Set Cover）	NP 完全	由顶点覆盖归约（每条边是一个元素，每个顶点是覆盖它的边集）
2-SAT	P（$O(n+m)$）	见 §7.6；这是"每子句 3 个文字"的临界点
树上独立集	P（$O(n)$ DP）	见 §12.4
二分图最大匹配	P（最大流）	见 §14.5
整数分解	NP ∩ coNP，**未知**是否 NP 完全	有短证书（两因子），但不知是否 NP 难
停机问题	**不可判定**	对角线论证；比 NP 难得多

16.7 经典反例与常见错误

错误	说明
归约方向反了	证"B 难"要从已知难的 A 归约**到** B（$A \leq_p B$）。写成 $B \leq_p A$ 只证明了"B 不比 A 难"，毫无用处
只证一个方向	缺 soundness 会把"归约过头"的构造漏掉（YES 实例被映射成 NO）
忘证 $B \in$ NP	只证 NP 难只说明"至少这么难"，可能不可判定（如停机问题也是 NP 难）
归约不是多项式的	例如枚举所有子集构造指数大小的图
把"NP 难"当"NP 完全"	NP 完全 = NP 难 + 属于 NP
认为"没有多项式算法"已被证明	未知。可能 P = NP；所有"NP 完全问题很难"的陈述都是**条件性**的（除非 P≠NP）

错误	说明
认为近似一定容易	有些问题的近似也是 NP 难的（如一般 TSP 的常数因子近似；PCP 定理给出更强的不可能性）

17. 第 16 章：应对 NP 难 —— 精确算法、FPT 与工程手段

17.1 问题动机

确认 NP 难之后，工程上还剩五条路（讲义的 "Coping with NP-completeness" 覆盖前几条，后两条是**推断**的现代实践补充）：

```
NP 难 ─┬─► 精确指数算法（2^n, n!）        ─► 小实例可用（TSP n≤25、MIS 稀疏图 n≈100）
       ├─► 固定参数算法 f(k)·poly(n)     ─► 参数 k 小时可用（顶点覆盖 O(2^k m)）
       ├─► 近似算法（近似比保证）         ─► 大规模、可接受误差（§20）
       ├─► 随机化/启发式（局部搜索）      ─► 无保证但快（§21）
       └─► 规约到 SAT/ILP/SMT            ─► 借成熟求解器（§17.5）
```

17.2 精确指数算法

思路：2^n 虽然指数，但 $n \le 40$ 时 $2^{40} \approx 10^{12}$ 在优化过的 C++ 里是"分钟级"；比 $n!$ 好太多。

问题	暴力	更好的精确算法
最大独立集	$O(2^n)$ 枚举子集	分支定界：选一个点 v，$\mathrm{MIS}(G) = \max(1 + \mathrm{MIS}(G - N[v]),\ \mathrm{MIS}(G - v))$，配合度数选点与剪枝，实践可处理 $n \approx 100$ 的稀疏图
3-SAT	$O(2^n)$ 枚举赋值	DPLL/CDCL：单元传播 + 学习子句，工业实例可处理百万变量（**推断**：现代 SAT 求解器已远超课程范围）
顶点覆盖	$O(2^n)$	分支：取边 (u,v)，递归 $\mathrm{VC}(G-u, k-1)$ 与 $\mathrm{VC}(G-v, k-1)$，$O(2^k \cdot m)$

分支定界（branch and bound）的三要素：分支（把问题拆成互斥子问题）、定界（用松弛/贪心算上下界）、剪枝（界劣于当前最好解则丢弃）。

17.3 固定参数可解（FPT）

观察：很多问题的难度集中在参数 k 上（"是否存在大小 $\le k$ 的顶点覆盖"），而不是 n 上。若算法是 $f(k) \cdot \mathrm{poly}(n)$，则当 k 小（实践中常见）时是可用的。

顶点覆盖的 FPT 算法：

```
VC(G, k):
    if k < 0: return False
    if E 为空: return True
    任取一条边 (u, v)
    return VC(G - u, k-1) or VC(G - v, k-1)
```

正确性：任何顶点覆盖必须包含 u 或 v（否则边 (u,v) 未被覆盖）。两种情形穷尽。□ **复杂度**：分支因子 2、深度 k，$O(2^k \cdot m)$。$k = 20$ 时约 $10^6 \cdot m$，完全可用。

对比：$O(2^k m)$ 与 $O(2^n)$ 的差别是"把指数放到参数上"。这是参数化复杂度（parameterized complexity）的核心思想。

17.4 近似算法与随机化（详见 §20、§21）

一句话预览：

- **顶点覆盖**：取极大匹配，输出其所有端点 → 2-近似，$O(m)$。
- **度量 TSP**：MST 双遍走 → 2-近似；Christofides → 1.5-近似。
- **集合覆盖**：贪心（每次选覆盖最多未覆盖元素的集合）→ $H(n) \approx \ln n$ 近似，且这个界**紧**（除非 P=NP，不能显著改进）。
- **最大割**：随机二分 → 期望 $1/2$ 的边被割，即 2-近似。
- **随机化**：随机快排、Karger 最小割、哈希。

17.5 工程实践：规约到 SAT / ILP / SMT

现代做法不是自己写指数算法，而是**把问题编码成 SAT 或 ILP，交给成熟求解器**。这本质上是"归约"的工程版：把 NP 难问题归约到 SAT（§16.5 的逆向使用），让几十年优化的 CDCL 求解器去吃。

手段	适用	工具举例（推断：非课程内容）
SAT 编码	组合约束、排课、验证	MiniSat、CaDiCaL、Kissat
ILP / MIP	有线性目标与整数变量	Gurobi、CPLEX、HiGHS、CBC
SMT	含算术/位向量/数组的约束	Z3、cvc5
CP（约束规划）	排列/调度类	OR-Tools CP-SAT
局部搜索/元启发式	大规模、允许近似	模拟退火、禁忌搜索、LNS

注意：这些工具**没有最坏情况保证**，可能在某些实例上跑到超时。工程上要设超时 + 可行解兜底（先用贪心拿一个解，再让求解器改进）。

17.6 经典反例

反例	说明
认为"2^n 一定不可行"	$n \le 25$ 时 $2^n \approx 3 \times 10^7$，配合剪枝很实用
认为"分支定界总有救"	最坏仍是指数；对抗性实例（如随机 3-SAT 在相变点附近）极难
把 FPT 当多项式	$O(2^k m)$ 在 $k = n/2$ 时仍是 $2^{n/2}$，k 大就退化
近似比 2 就"够用"	近似比是**最坏情况**保证；实际误差可能远小于 2，也可能刚好卡在 2
忽略"近似也是 NP 难"	一般（非度量）TSP 不存在任何常数因子近似（除非 P=NP）

18. 专题一：证明方法论

CS170 的考试有一半分数在"证明"上。本章把散落在各章的证明技巧抽成可复用的模板。**核心原则：一个证明的价值在于"读者能检查每一步"，而不是"结论听起来对"。**

18.1 循环不变式（Loop Invariant）

适用：迭代算法（BFS、Dijkstra、Kruskal、插入排序、并查集）。

三段式模板：

> **不变式 P**：在每轮循环开始时，下列命题成立：…… **初始化**：进入循环前 P 成立。（通常是一行验证） **保持**：若某轮开始时 P 成立，且本轮执行了循环体，则下轮开始时 P 仍成立。（这是证明的主体） **终止**：循环结束时 P 与"循环条件为假"共同蕴含正确性。

例（Dijkstra）：P = "已出队的顶点集合 S 中，每个 v 的 `dist[v]` 等于真正的最短距离；且对 S 外每个 v，`dist[v]` 是只经过 S 内顶点的最短距离"。

- 初始化：$S = \emptyset$，`dist[s]=0`，其余 ∞，成立。
- 保持：取出堆中最小者 u，用反证法证 `dist[u]` 正确（§8.2 的论证），再对所有出边松弛。
- 终止：$S = V$，所有距离正确。

常见错误：不变式写得**太弱**（无法推出结论）或**太强**（保持步证不出来）。判断标准：终止时能否直接推出目标命题。若不能，不变式不够强；若保持步卡住，不变式过强。

18.2 交换论证（Exchange Argument）

适用：贪心算法的正确性（区间调度、Huffman、MST、调度）。

四步模板：

1. 设 G 是贪心解，O 是**任一**最优解，且在所有最优解中与 G 的**公共前缀最长**（这个"最长"是让反证成立的关键技巧）。

2. 找第一个分歧位置 i：$g_i \neq o_i$。
3. 用贪心准则证明可以把 o_i **替换**成 g_i，且解**不变差**、仍然可行。
4. 得到与 O 同样优但前缀更长的解，矛盾。

关键点：第 3 步必须验证**可行性**（替换后不违反约束）和**不劣性**（目标值不下降）。这两件事缺一不可。

例（区间调度，§9.2）：由"最早结束"得 $e(g_i) \le e(o_i)$；替换后 g_i 与前后都兼容（前面靠贪心选择，后面靠 $e(g_i) \le e(o_i) \le s(o_{i+1})$）。

例（MST 割性质，§10.2）：交换的是"环上的另一条边"，用 $w_e \le w_{e'}$ 保证不劣。

18.3 摊还分析（Amortized Analysis）

适用：操作序列的总代价远小于"单次最坏代价 × 次数"的场景。

为什么需要：单次操作可能很贵（动态数组扩容 $O(n)$、并查集路径压缩可能走很长），但**平均到每次**很便宜。摊还分析给的是**最坏情况下的平均**（不是期望，不涉及概率），比平均情况分析更强。

三种方法：

方法	思路	例
聚合分析（aggregate）	直接算 n 次操作的总代价，除以 n	二进制计数器
记账法（accounting）	给每种操作"预付"虚拟货币，贵操作花存款	动态数组扩容
势能法（potential）	定义势函数 Φ，摊还代价 = 实际代价 + $\Delta\Phi$	并查集、伸展树

18.3.1 聚合分析：二进制计数器

一个 k 位计数器，每次 `+1` 翻转若干位。最坏一次翻转 k 位，n 次最坏 $O(nk)$。但**总翻转次数**：第 0 位每次翻，第 1 位每 2 次翻，第 i 位每 2^i 次翻，故

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i \ge 0} 2^{-i} = 2n$$

所以 n 次操作总代价 $O(n)$，**摊还** $O(1)$。

18.3.2 记账法：动态数组（dynamic table）

容量为 m、已存 m 个元素时插入，需扩容到 $2m$ 并复制 m 个元素，代价 $O(m)$。

记账：每次普通插入收 3 元——1 元付本次插入，2 元存起来（每个元素存 2 元）。扩容时复制 m 个元素需要 m 元，而存款恰好有 $2m$ 元（每个旧元素 2 元），足够。故 n 次插入总代价 $\le 3n$，**摊还** $O(1)$。

18.3.3 势能法：定义与例

$$\text{摊还代价}_i = \text{实际代价}_i + \Phi(D_i) - \Phi(D_{i-1})$$

求和望远镜化：$\sum \text{摊还} = \sum \text{实际} + \Phi(D_n) - \Phi(D_0) \ge \sum \text{实际}$（若 $\Phi \ge \Phi_0$）。所以"摊还上界"蕴含"总实际上界"。

例（动态数组）：令 $\Phi = 2 \cdot \text{size} - \text{capacity}$（size 为元素数，capacity 为当前容量）。插入不扩容时 $\Delta\Phi = 2$，摊还 $= 1 + 2 = 3$；扩容时实际 $= \text{size} + 1$，$\Delta\Phi = 2 - \text{size}$（容量翻倍抵消），摊还 $= (\text{size} + 1) + (2 - \text{size}) = 3$。**恒为 3**，故 $O(1)$ 摊还。

例（并查集）：路径压缩 + 按秩合并，m 次操作的摊还代价 $O(m\,\alpha(n))$，α 是反 Ackermann 函数，对任何可想象的 n 都 ≤ 4。证明用势能法（Φ = 各节点"秩 − 势"的对数之和），是摊还分析里最复杂但最漂亮的例子；CS170 只要求记住结论与"为什么近似常数"。

18.4 归纳法与强归纳

适用：递归算法正确性、DP 正确性、组合恒等式。

模板：对"子问题规模"归纳（不是对时间）。DP 的归纳对象是**子问题 DAG 的拓扑序**：假设所有依赖的子问题都已正确，证当前子问题正确。

例（编辑距离，§11.4）：对 $i + j$ 归纳。$E(i, j)$ 的三个候选项分别对应"最后一次操作是删除/插入/替换"，必有一种是最优解的最后一步。

陷阱：归纳假设必须**覆盖所有被用到的子问题**。若转移用到 $E(i-1, j)$ 与 $E(i, j-1)$，归纳要对 $i + j$ 做（不能只对 i 做）。

18.5 反例证明（Disproof）

适用：证伪"某贪心最优"、"某界更紧"、"某命题对所有图成立"。

模板：给出**一个具体的、足够小的**实例，算出贪心/命题的值与最优值，指出不等。

要求：实例要**小到能手工验证**（通常 3–6 个元素），且要说明"为什么这构成反例"（即两个值确实不同）。

例（0/1 背包贪心失败，§12.1）：$W = 10$，物品 $(6,6),(5,5),(5,5)$。单位价值贪心得 6，最优 10。三行写完，无可辩驳。

例（区间调度"最早开始"失败，§9.2）：$[0,10]$ 与 $[1,2],[2,3]$。最早开始选 $[0,10]$ 得 1，最优 2。

19. 专题二：归约的构造方法

19.1 归约的四种常见形态

形态	做法	例
恒等/参数变换	不改结构，只改参数	独立集 ↔ 顶点覆盖（$k \leftrightarrow n-k$）
补图/取反	把"不相邻"变"相邻"	独立集 → 团
gadget 构造	为每个变量/子句造一个小部件，用边/路径连接	3-SAT → 独立集、3-着色
编码/数位化	把组合结构编码成数	3-SAT → 子集和

19.2 构造归约的五个步骤

1. **明确两个问题的"输入–输出"格式**。先把判定版写清楚（"是否存在 $\le k$ 的……"），优化版先转判定版。
2. **找"信息载体"**：源问题的哪些信息必须被目标问题"记住"？变量？子句？边？用目标问题的什么结构承载（顶点、边、数位、路径）？
3. **设计 gadget**：为每个局部约束造一个小图/小数，使"局部可满足 $\iff$ 局部结构有解"。
4. **连接 gadget**：用边或共享变量表达**全局一致性**（如"x 与 $\neg x$ 不能同真"）。
5. **双向证明 + 规模检查**：证两个方向，算清 $f(x)$ 的大小是多项式。

19.3 gadget 设计的三个常用技巧

技巧	作用	例
三角形/团	强制"至多选一个"	3-SAT → IS 的每个子句三角形
强制对（两个顶点连边）	强制"至少选一个"	顶点覆盖里的边
链式变量 gadget	让"真/假"对应"走左/走右"	3-着色、哈密顿回路

设计检验清单：

- 局部 gadget 是否**只**受本变量/子句影响？（避免意外耦合）
- 全局约束是否**只**通过设计好的接口传递？
- 目标参数 k 是否算对？（最常见的 off-by-one：k = 子句数、$n-k$、$3m$ 等）
- 是否存在"退化解"（不满足局部约束也能凑出目标值）？——用 soundness 方向检查。

19.4 归约速查表（完整版见速查文档）

从	到	构造要点	参数
SAT	3-SAT	长子句拆链，短子句补哑变量	子句数 $O(\text{长度})$
3-SAT	独立集	每子句一个三角形 + 互补文字连边	$k = m$（子句数）
独立集	顶点覆盖	同图，取补集	$k' = n - k$
独立集	团	补图	k 不变

从	到	构造要点	参数
顶点覆盖	集合覆盖	元素=边，集合=顶点覆盖的边	k 不变
3-SAT	子集和	数位编码 + 每子句两个哨兵	目标和 t
子集和	0/1 背包	价值=重量	容量=目标和
哈密顿回路	TSP	有边权 1，无边权 2，界 n	n
3-SAT	3-着色	基三角形 + 变量颜色对 + OR-gadget	固定 3 色

19.5 归约的三个经典错误

1. **方向反了**：见 §16.7。记住口诀"**从难到易**"：已知难的放左边，要证的放右边。
2. **只证一半**：缺 soundness 会让"目标问题有解但源问题无解"的情形溜过去，归约不成立。
3. **规模爆炸**：gadget 数量必须是多项式的。例如"为每个子集造一个顶点"是 2^n 个顶点，不是多项式归约（这是"伪多项式"与"多项式"的分界）。

19.6 判定版 vs 优化版

版本	形式	归约时的处理
判定版	"是否存在 $\le k$ 的解"	NP 完全性的标准载体
优化版	"求最小/最大值"	NP 难（不是 NP 完全，因为输出不是 YES/NO）
搜索版	"给出一个最优解"	用判定版 + 二分/自归约（self-reducibility）求解

自归约：若判定版有多项式算法，优化版也能多项式求解（对 k 二分，或逐个固定变量）。所以"判定难 $\iff$ 优化难"。

20. 专题三：近似算法

20.1 定义与记号

对最小化问题，算法 $\mathcal{A}$ 是 **α-近似**（$\alpha \ge 1$）若对所有实例，

$$\mathcal{A}(I) \le \alpha \cdot OPT(I)$$

（最大化问题改为 $\mathcal{A}(I) \ge OPT(I)/\alpha$。）$\alpha$ 称为**近似比**（approximation ratio）。

注意：近似比是**最坏情况**保证；α 越接近 1 越好；$\alpha = 1$ 即精确算法。

20.2 顶点覆盖的 2-近似

算法：

```
ApproxVC(G):
    C = []
    while E 非空:
        任取一条边 (u, v)
        C.add(u); C.add(v)
        删除所有与 u 或 v 关联的边
    return C
```

即"取一个**极大匹配**（maximal matching，不是最大匹配），输出其所有端点"。

定理：$|C| \le 2 \cdot OPT$。

证明：算法选出的边两两不共享端点（匹配），设选了 k 条边。任何顶点覆盖必须覆盖这 k 条边，而它们互不相交，故至少需要 k 个顶点：$OPT \ge k$。算法输出 $2k$ 个顶点，故 $|C| = 2k \le 2\,OPT$。□

紧性：完全二分图 $K_{n,n}$ 上，$OPT = n$（取一侧），算法可能选 $2n$ 个（若每次都选到不同两侧的点）——比值恰好 2。**所以 2 不能再改进（用这个算法）**。

复杂度：$O(n+m)$。

为什么不能做到 $2-\epsilon$：一般图上顶点覆盖的最佳已知近似比约 2（$2-\epsilon$ 需要假设 P≠NP 之外的更强假设；Unique Games 猜想下 2 是最优）。**推断**：这属于近似难度理论，课程不要求。

20.3 度量 TSP 的 2–近似与 Christofides 1.5–近似

前提：距离满足**三角不等式** $d(u,w) \le d(u,v) + d(v,w)$（度量 TSP）。一般 TSP 没有常数因子近似（除非 P=NP）。

20.3.1 2–近似（MST 双遍）

```
ApproxTSP2(G):
    T = MST(G)
    从任意点出发，沿 T 做 DFS，记录访问序列（每个点可能多次）
    按首次出现去重，得到哈密顿回路
    return 该回路
```

定理：$cost \le 2 \cdot OPT$。

证明：DFS 遍历每条树边恰好两次，故走遍所有点的总长 $= 2 \cdot cost(MST)$。由三角不等式，去重（抄近路）不会增加长度，故回路长 $\le 2 \cdot cost(MST)$。又：删去最优 TSP 回路的任一条边得到一棵生成树，故 $cost(MST) \le OPT$。合并得 $cost \le 2\,OPT$。□

20.3.2 Christofides 1.5–近似

观察：MST 双遍走"每条边两次"太浪费。若能把奇度顶点配对，使欧拉回路只走一次多。

```
Christofides(G):
    T = MST(G)
    O = T 中度为奇数的顶点集（|O| 必为偶数，握手定理）
    M = O 上的最小权完美匹配
    在 T + M 上找欧拉回路（所有顶点度为偶数）
    沿欧拉回路走，按首次出现去重 → 哈密顿回路
```

定理：$cost \le \frac{3}{2} OPT$。

证明：$cost(T) \le OPT$（同 2–近似）。关键看 $cost(M)$：把最优 TSP 回路在 O 上的顶点按回路顺序取出，得到 O 上的一个**偶环**（两条交错匹配的并），每条匹配的长度 $\le OPT/2$；最小完美匹配不劣于其中任一条，故 $cost(M) \le OPT/2$。欧拉回路总长 $= cost(T) + cost(M) \le \frac{3}{2} OPT$，去重不增（三角不等式）。□

紧性：存在实例使比值趋近 3/2（**推断**：标准构造是"两个团 + 一条细长连接"，课程不要求构造细节）。

复杂度：MST $O(m \log n)$ + 最小权完美匹配（Blossom 算法，$O(n^3)$）。

20.4 集合覆盖的 $\ln n$ 近似

问题：全集 U（$|U| = n$），一族子集 $S_1, \dots, S_m$，选最少的子集覆盖 U。

```
GreedySetCover(U, S):
    R = U                       # 未覆盖元素
    C = []
    while R 非空:
        选使 |S_i ∩ R| 最大的 S_i
        C.add(S_i); R = R \ S_i
    return C
```

定理：$|C| \le H(n) \cdot OPT$，其中 $H(n) = 1 + \frac{1}{2} + \cdots + \frac{1}{n} \le \ln n + 1$。

证明（记账/收费论证）：设贪心第 t 步选了 S_t，覆盖了 k_t 个新元素。把"每单位覆盖成本" $1/k_t$ 摊到这些新元素上。设最优解有 OPT 个集合，则任一时刻存在一个集合至少覆盖 $|R|/OPT$ 个剩余元素，故贪心选的集合满足 $k_t \ge |R_t|/OPT$，即每个新元素被摊到的成本 $\le OPT/|R_t|$。

按"元素被覆盖时剩余规模"分组：剩余规模从 n 降到 1，第 j 个元素至多被摊到 OPT/j。总成本

$$\sum_{j=1}^{n} \frac{OPT}{j} = OPT \cdot H(n)$$

而总成本又等于 $\sum_t 1 = |C|$（每步摊 1）。故 $|C| \le H(n) \cdot OPT$。□

紧性：存在实例使贪心达到 $\Theta(\log n)$ 倍（**推断**：标准构造是"大集合 + 大量小集合"，课程不要求）；且在标准复杂性假设下集合覆盖不能近似到 $(1-\epsilon)\ln n$（Feige 1998，超出课程）。

复杂度：$O(mn)$（每步扫描所有集合）。

20.5 近似算法速查

问题	近似比	算法	复杂度	下界/紧性
顶点覆盖	2	极大匹配取端点	$O(n+m)$	紧（$K_{n,n}$）
度量 TSP	2	MST 双遍	$O(m \log n)$	紧
度量 TSP	3/2	Christofides	$O(n^3)$	紧
集合覆盖	$H(n) \le \ln n + 1$	贪心	$O(mn)$	紧（至 $\ln n$）
最大割	1/2（即 2–近似）	随机二分	$O(n+m)$	0.878 有更好的（Goemans–Williamson SDP）
0/1 背包	$1-\epsilon$（FPTAS）	按价值缩放 DP	$O(n^3/\epsilon)$	有 FPTAS
一般 TSP	无常数因子	—	—	除非 P=NP
最大独立集	$n^{1-\epsilon}$ 都难	—	—	极难近似

20.6 经典反例

反例	说明
对一般 TSP 用 MST 双遍	三角不等式不成立时，去重"抄近路"可能**增加**长度（绕路 shortcut 不存在），2–近似证明失效；实际上一般 TSP 无常数近似
认为"贪心集合覆盖"总能到 $\ln n$	$\ln n$ 是上界；某些实例贪心恰好差 $\ln n$ 倍，但多数实例远好于界
用"取整 LP"当近似算法	取整后可能不可行；需要设计 rounding 方案并证明比值

21. 专题四：随机算法

21.1 为什么要随机

随机化把"最坏情况"从对手手里拿走。确定性算法的最坏输入可以被构造；随机算法的输入在算法掷骰子**之前**就固定了，对手无法针对随机选择构造实例。讲义对随机算法运行时间的定义很精确：**对手看到算法描述、选一个坏实例，然后算法掷骰子**；因此运行时间是随机变量，我们讨论**期望运行时间**。

类型	保证	例
Las Vegas	结果**总是正确**，运行时间是随机变量	随机快排、QuickSelect
Monte Carlo	运行时间确定，结果**以一定概率错**	Karger 最小割（截断版）、Miller–Rabin 素性测试

21.2 随机快排（Randomized Quicksort）

```
RandomizedQuicksort(A):
    if |A| <= 1: return A
    均匀随机选枢轴 p
    划分成 L (<p), E (=p), R (>p)
    return RandomizedQuicksort(L) + E + RandomizedQuicksort(R)
```

定理：期望比较次数 $\le 2n\ln n + O(n) = O(n\log n)$。

证明（指示器变量）：设排序后元素 $z_1 < \cdots < z_n$，$X_{ij} = 1$ 表示 z_i, z_j 被比较。二者被比较 $\iff$ 在包含 $\{z_i..z_j\}$ 的子问题中，第一个被选为枢轴的是 z_i 或 z_j，概率 $\frac{2}{j-i+1}$。于是

$$E[X] = \sum_{i<j} \frac{2}{j-i+1} = \sum_{d=1}^{n-1} \frac{2(n-d)}{d+1} < 2n\sum_{d=1}^{n-1}\frac{1}{d} = O(n\log n)$$

更精确地，$E[X] = 2(n+1)H_n - 4n \approx 2n\ln n$。□

最坏情况仍是 $O(n^2)$（每次枢轴都极值）。但**期望**对任意输入都是 $O(n\log n)$——注意这里的期望只对算法的随机性取，**不对输入分布做假设**，这是随机算法比"平均情况分析"更强的地方。

21.3 Karger 最小割（Karger's Contraction）

问题：无向图 G 的最小割（删去最少的边使图不连通）。

收缩（contraction）：选一条边 (u,v)，把 u, v **合并**成一个顶点，保留所有其它边（出现自环则删除，重边保留为多重边）。收缩一次顶点数减 1，割的大小不变或变大（**不会变小**）。

```
Karger(G):
    while 顶点数 > 2:
        均匀随机选一条边 e
        收缩 e（合并两端，删自环，保留重边）
    return 剩下的两条顶点之间的所有边（即一个割）
```

成功概率分析：

> **引理**：设最小割大小为 k，则 $m \ge nk/2$（每个顶点的度 $\ge k$，握手定理）。 **引理**：收缩过程中，若当前图有 n' 个顶点，随机边属于该最小割的概率 $\le \frac{k}{n'k/2} = \frac{2}{n'}$。

（第二条用"每个顶点度 $\ge k$ 在收缩后仍成立"：收缩不降低最小割，故当前图最小割 $\ge k$，边数 $\ge n'k/2$。）

于是最小割在全部 $n-2$ 次收缩中都没被选中的概率

$$\Pr[\text{成功}] \ge \prod_{n'=n}^{3}\left(1-\frac{2}{n'}\right) = \frac{(n-2)(n-3)\cdots 2\cdot 1}{n(n-1)\cdots 4\cdot 3} = \frac{2}{n(n-1)} \ge \frac{2}{n^2}$$

重复 $O(n^2\log n)$ 次，失败概率 $\le (1-2/n^2)^{n^2\log n} \le e^{-2\log n} = 1/n^2$。每次运行 $O(n^2)$（邻接矩阵 + $n-2$ 次收缩），总 $O(n^4\log n)$。**这是多项式时间算法**（虽然慢；有 $O(n^2\log^3 n)$ 的改进版）。

经典反例：Karger 单次运行的成功率只有 $\Theta(1/n^2)$，**不重复就基本一定错**。这与"随机算法"的常见误解相反——随机化不是"跑一次碰运气"，而是"用概率放大把错误率压到可忽略"。

21.4 哈希（Hashing）

目标：n 个键映射到 m 个桶，期望 $O(1)$ 查找。

通用哈希族（universal family）：$\mathcal{H}$ 是通用的，若对任意 $x \ne y$，$\Pr_{h\in\mathcal{H}}[h(x)=h(y)] \le 1/m$。

构造：取素数 $p >$ 所有键，$h_{a,b}(x) = ((ax+b) \bmod p) \bmod m$，$a \in \{1..p-1\}, b \in \{0..p-1\}$。

定理：若 h 从通用族均匀选取，则一次查找的期望代价 $O(1+n/m)$。

证明：设查 x，x 与 y 冲突的概率 $\le 1/m$，由期望线性性，冲突数期望 $\le (n-1)/m$。□

为什么需要随机化：**对手可以构造"全部冲突"的输入**（若哈希函数固定且公开）。随机选 h 后，对手在选输入时不知道 h，冲突数期望被 n/m 控制。这是"随机化对抗自适应对手"的又一例。

负载均衡（balls into bins）：n 个球随机扔进 n 个桶，最大负载以高概率为 $\Theta(\log n/\log\log n)$。这个结论解释了"链地址法的链表长度"和"分布式系统的最慢节点"。

22. 与已有笔记的衔接

22.1 ← CS70：离散与概率

CS70 内容	在 CS170 的用处	备注
归纳法	所有正确性证明	强归纳用于 DP（对子问题 DAG 拓扑序）
图论（度、路径、连通、树）	第 6、9 章	树的等价定义（连通无环 $\iff$ $n-1$ 条边）在 MST 证明里直接用
期望的线性性	随机快排、Karger、哈希	**不需要独立性**，这是它好用的原因
条件概率	QuickSelect 的"好枢轴"概率	$\Pr[\text{好}] \geq 1/2$
Markov / Chebyshev / Chernoff	概率放大、负载均衡	$O(\log n)$ 次重复压错误率
模运算	通用哈希的 $ax+b \bmod p$	需要 p 为素数

不要重复学：CS70 已讲的计数、条件概率定义、期望定义，本笔记默认你已掌握。

22.2 ← CS229：凸优化与对偶

CS229 内容	与 CS170 的对应
凸集、凸函数	LP 的可行域是凸多面体（§13.2）
拉格朗日乘子	LP 对偶变量 y 就是乘子（§15.2）
KKT 条件	**互补松弛**是 KKT 在 LP 上的特化（§15.4）
SVM 的对偶推导	与 §15.2 的"max → min 转置"完全同构：原问题变量 w，对偶变量 α
梯度下降	与 LP 无关；但**内点法**用牛顿方向，属于二阶方法

一句话：CS229 教你"怎么解"，CS170 教你"为什么对偶最优值相等"以及"对偶的经济解释（影子价格）"。

22.3 ← 15-445：join 顺序与贪心/DP

数据库概念	CS170 对应	差异
Join order 选择	**链矩阵乘的 DP**（§12.2）：$C(i,j) = \min_k C(i,k) + C(k+1,j) + \text{cost}$	结构完全相同，代价模型换成基数估计
Selinger 优化器（System R）	上述 DP + 剪枝（只保留每子集的最优计划）	本质是 Held–Karp 风格的子集 DP
贪心 join 顺序	§9 贪心	表多时用贪心/遗传算法（如 PostgreSQL 的 GEQO）
选择率估计	无直接对应	这是 DB 特有的统计问题，误差会毁掉 DP 的最优性
索引选择	§13 LP（**推断**：某些系统用 ILP 做索引推荐）	工业界多用启发式

关键洞察：**"子集 DP + 剪枝"是跨领域的通用模式**——TSP（§12.3）、join 顺序、以及很多编译器优化问题都是它的实例。

23. 复习清单与自测题

23.1 复习清单（按"必须能当场写出"分级）

A 级（不写出来就等于没学）

- [ ] $O/\Omega/\Theta$ 的形式定义 + $0.1n^2 \notin O(n)$ 的反证；Master 定理三情形 + 两个用不了的例子（$n \log n$ 项、$\sqrt{n}$ 递归）。

- [ ] Karatsuba 的恒等式与 $T(n)=3T(n/2)+O(n)$ 的递归树求和；区间调度的交换论证（含"前缀最长"技巧）。
- [ ] Bellman–Ford 的 $n-1$ 轮归纳 + 负环检测。
- [ ] Dijkstra 伪代码 + 非负权必要性 + 负权反例。
- [ ] MST 割性质证明 + Kruskal/Prim 伪代码。
- [ ] 0/1 背包转移 + 逆序枚举的原因 + 伪多项式解释。
- [ ] 编辑距离转移 + 三情形穷尽性。
- [ ] 最大流最小割定理的三步证明。

B 级（能复述思路与结论）

- [ ] Strassen 的 7 次乘法与 $O(n^{2.807})$；QuickSelect 的"好枢轴"论证与 Median-of-Medians 的 $3n/10$ 引理。
- [ ] 最近点对的"7 邻居"引理与 $O(n\log n)$；Kosaraju 两遍 DFS 的正确性直觉。
- [ ] Huffman 的两个引理（兄弟叶 + 最优子结构）；Held–Karp 的 $O(n^2 2^n)$ 状态定义。
- [ ] 树上独立集 DP 的两个状态。
- [ ] LP 对偶的构造规则 + 弱对偶一行证明。
- [ ] 归约链：3-SAT → IS → VC → Clique，以及 3-SAT → 子集和。
- [ ] 顶点覆盖 2-近似、Christofides 3/2、集合覆盖 $\ln n$ 的证明骨架。

23.2 自测题（10 题，答案紧随）

Q1. 判断：$T(n)=2T(n/2)+n\log n$ 可以用 Master 定理直接得到 $\Theta(n\log n)$。 **A1.** 错。真值是 $\Theta(n\log^2 n)$。$f(n)=n\log n$ 带对数因子，不满足 $f(n)=\Theta(n^d)$ 的"纯多项式"形式；若机械套 $a=b^d=2$ 就会错得 $\Theta(n\log n)$。正确做法是递归树：第 t 层有 2^t 个节点、每个规模 $n/2^t$，本层开销 $n(\log n-t)$，故 $T(n)=\sum_{t=0}^{\log n} n(\log n-t)=\Theta(n\log^2 n)$。

Q2. 给一个图，使 Dijkstra 输出错误的最短距离，并说明错在哪一步。 **A2.** $s\to a$ 权 2，$s\to b$ 权 5，$b\to a$ 权 -4。Dijkstra 先取出 a（`dist=2`）并标记确定，之后 b 松弛出 `dist[a]=1` 却不被采纳。错在"边权非负 ⟹ 先出队者的距离已最优"这一步失效。

Q3. 0/1 背包的 $O(nW)$ 算法是否证明背包属于 P？ **A3.** 不。W 是数值，输入长度是 $\log W$；$O(nW)=O(n2^{\log W})$ 对输入规模是指数的（伪多项式）。背包（判定版）是 NP 完全的。

Q4. 证明：若 G 有大小 k 的独立集，则 G 有大小 $n-k$ 的顶点覆盖。 **A4.** 设 S 为独立集，$|S|=k$。对任意边 (u,v)，若两端都不在 $V\setminus S$，则 $u,v\in S$，与 S 独立矛盾。故 $V\setminus S$ 覆盖所有边，大小 $n-k$。

Q5. 最小生成树的割性质中，若最轻边不唯一，结论还成立吗？ **A5.** 成立。证明只用到 $w_e\le w_{e'}$（非严格），交换后 $w(T')\le w(T)$，由 T 最优得等号，T' 仍是 MST。**结论是"存在 MST 含 e"，不是"所有 MST 都含 e"。**

Q6. 写出最大流最小割定理的证明提纲（三步）。 **A6.** ①弱对偶：任意流 $\le$ 任意割（用守恒式求和 + 容量约束）；②若无增广路径，取残量图中 s 可达集 S，则跨割边全饱和、反向边全零，故 $|f|=c(S,T)$；③合并得 $\max|f|=\min c(S,T)$，且 FF 终止时的流最大。

Q7. 归约 $A\le_p B$ 中，若 A 是 NP 完全的，能推出 B 是 NP 完全吗？ **A7.** 只能推出 B 是 **NP 难**。要成为 NP 完全还需 $B\in$ NP。反例：停机问题是 NP 难的，但不在 NP 中。

Q8. 顶点覆盖的 2-近似算法为什么不能直接用于"最小支配集"？ **A8.** 因为证明依赖"匹配的边互不相交，任何覆盖至少需要每条边一个端点"，这个下界对支配集不成立（一条边可以被一个端点"支配"，但支配集的要求是"每个顶点被选中或与选中点相邻"，结构不同）。**近似算法的证明与其组合结构强绑定，不能跨问题套用。**

Q9. Christofides 算法中，为什么奇度顶点集 O 的大小一定是偶数？ **A9.** 握手定理：所有顶点度数之和 $=2|E|$ 为偶数；偶度顶点贡献偶数，故奇度顶点个数必为偶数，否则总和的奇偶性矛盾。

Q10. 随机快排的期望 $O(n\log n)$ 是否依赖"输入随机"？ **A10.** 不依赖。期望只对算法内部的随机枢轴取，对**任意固定输入**都成立。这正是随机算法比平均情况分析强的地方。

24. 参考来源

抓取日期：2026-11-01（America/Chicago）。以下均为公开可访问页面/PDF；课程内容可能随学期变动，以课程当学期主页为准。

24.1 课程页面

内容	URL	状态
课程主页（Fall 2026 日程、讲次、Readings、作业/讨论链接）	https://cs170.org/	✅ 成功（HTTP 200，145 KB）
Policies（先修、评分、考试日期、作业政策）	https://cs170.org/policies/	✅ 成功
Calendar（Google Calendar 嵌入，无静态文本）	https://cs170.org/calendar/	⚠️ 页面可访问，但内容为 JS 嵌入日历，只能读到标题；课程日程改从主页表格取得
Resources（往年真题、录播、作业规范）	https://cs170.org/resources/	✅ 成功；"Past Exams" 指向 Google Drive 外链，本环境无法解析
Discussions（讨论课时间地点）	https://cs170.org/discussions/	✅ 成功

24.2 讲义 PDF（18 份，前缀 `https://cs170.org/assets/lec/`）

全部 ✅ 抓取并 `pdftotext -layout` 解析成功：`01_intro_filled.pdf`、`02_divide-conquer-I_filled.pdf`、`03_divide-conquer-II_filled.pdf`、`04_divide-conquer-III_filled.pdf`、`lec-5-filled.pdf`（图分解）、`lec-6-filled.pdf`（有向图/SCC）、`lec-7-paths-in-graphs.pdf`、`08_greedy_filled.pdf`、`09_greedy_blank.pdf`（仅 blank 版）、`10_Huffman_Kruskal_blank.pdf`（仅 blank 版）、`11_DP1.pdf`、`12_DP2.pdf`、`13_DP3_class.pdf`、`14_DP4_blank.pdf`、`lec-15.pdf`（LP）、`lec-16.pdf`（最大流）、`lec-17-duality.pdf`、`lec-18.pdf`（P/NP，扫描页多、文字层稀疏）。

> 失败项：`09_greedy.pdf` 与 `10_Huffman_Kruskal.pdf` 均返回 **404**，主页只提供对应 `_blank` 版本；这两讲的内容我依据 blank 版与 DPV 相应章节补全。

24.3 作业与讨论课（抽样核验）

内容	URL	状态
Homework 5（含"三腿赛跑"匹配类问题）	https://cs170.org/assets/pdf/hw05.pdf	✅
Homework 9（LP 建模与对偶，含饮食配方题）	https://cs170.org/assets/pdf/hw09.pdf	✅
Discussion 9（最大流/最小割）	https://cs170.org/assets/pdf/dis09.pdf	✅
Homework 13 / 14 / Disc 11 / Disc 14	同名 `.pdf`	❌ 404（主页列表引用，但文件暂未发布或路径变动）

24.4 教材（DPV 公开章节，作者主页）

Dasgupta, Papadimitriou, Vazirani, *Algorithms*，各章 PDF 前缀 `https://people.eecs.berkeley.edu/~vazirani/algorithms/`：`chap0.pdf`（§0 Prologue：渐近与概率复习）、`chap1.pdf`（§1 数论算法）、`chap2.pdf`（§2 分治）、`chap3.pdf`（§3 图分解）、`chap4.pdf`（§4 图中的路径）、`chap5.pdf`（§5 贪心）、`chap6.pdf`（§6 动态规划）、`chap7.pdf`（§7 线性规划与归约）、`chap8.pdf`（§8 NP 完全问题）、`chap9.pdf`（§9 应对 NP 完全性）。全部 ✅ 可访问。

24.5 声明

- 本笔记的**定理陈述、算法伪代码、反例**均由我依据上述公开材料重写与组织；讲义的原始图表、作业题目原文**未复制**。
- 标注"**推断**"的内容（如工程实践、数值稳定性、PCP/ETH、工业工具）**不属于**课程材料，是我基于领域常识的补充，读者应自行核实。
- 涉及评分比例、考试日期、授课教师的信息全部来自 2026-11-01 抓取的 Policies 页与主页；**若你不在 Fall 2026 学期，请以课程当学期主页为准**。
- 本笔记不构成对课程材料的再授权；课程材料版权归 UC Berkeley 及各位作者。